# Intelligent Internal Developer Platforms

An AI-Native Approach to Strategic Software Delivery and Governance

**Prince Victor Orjiugo**

Master's Degree Project in
Strategic Information Systems Management
Spring term 2026
Supervisor: Dr. Gideon Mekonnen Jonathan
Department of Computer
and Systems Sciences




Stockholm
University

# Abstract

**Introduction.** Modern software delivery is increasingly mediated by Internal Developer Platforms, yet code, logs, incident history, and architectural intent often remain fragmented across tools. This thesis examines the resulting gap between code produced and code understood through the conceptual lens of epistemic stress, grounded in knowledge management theory, and explores the proposition that artificial intelligence could serve as foundational platform infrastructure rather than as an added feature.

**Research Question.** The study asks: How can an AI-native Internal Developer Platform be designed to build unified, context-specific intelligence from software lifecycle data, and what are its effects on software delivery, developer experience, and governance?

**Method.** Following Design Science Research, the study develops RootOps, an open-source platform demonstrated against a production-grade application in eight expert-interview sessions. Interview data were analysed through reflexive thematic analysis; the evidence is interpretive and perception-based, and claims are bounded accordingly.

**Results.** RootOps instantiates four design principles: a shared semantic substrate across code and logs, system-specific learning over a bounded scope, AI as the foundational layer, and governance as a continuous embedded property. Expert evaluation yielded ten themes, including recognition of epistemic stress in practitioners' own terms, indications that the IIDP pattern is emerging independently in industry, critique of vector-based retrieval, and unresolved questions about architectural scope and developer-profiling ethics.

**Discussion.** The findings suggest that the thesis's contribution is an empirically grounded articulation of the Intelligent Internal Developer Platform pattern, accompanied by a reproducible reference artifact and an account of practitioner evaluation. The thesis separates durable principles from contestable implementation choices and identifies retrieval substrate, boundary drawing, and governance as future research problems.

# Acknowledgements

Writing a thesis is rarely a solitary endeavour; it is a journey made possible through the shared wisdom, patience, and belief of an entire community.

I am deeply grateful to my supervisor, Dr. Gideon Mekonnen Jonathan, for his guidance, patience, and consistently sharp feedback throughout this work. His willingness to engage critically with both the research framing and the artifact itself shaped this thesis at every stage.

I would like to thank the eight practitioners who gave their time to the expert-interview sessions reported in this thesis. Their candour, seniority, and willingness to challenge the design made the empirical contribution possible. They are anonymised here, but their forward-thinking and insights are visible throughout Chapter 5.

My work is heavily informed by the brilliant minds around me. I am grateful to the community at Stockholm AI for the conversations and connections that informed this research. A special thanks belongs to the 2hero community and the DORA Google groups; your collective wisdom, open-source spirit, and relentless pursuit of better engineering practices have been an invaluable wellspring of inspiration.

A journey of this magnitude requires a strong foundation of personal support, and I am blessed with an extraordinary one. Thank you to my friends, Anthonia, Jady, and Malsha, for your unwavering encouragement, your patience over the months this work took to complete, and for always keeping me grounded. To you, thank you for your steadfast support and for being a continuous pillar of strength when the work demanded it most.

Finally, the deepest well of my gratitude belongs to my family, and above all, my mother, Mrs. Francisca Orjiugo. You have always been my greatest inspiration. Your sacrifices, resilience, and endless belief in my potential laid the foundation for every milestone I reach. This achievement is as much yours as it is mine.

Dare to dream.

# Abbreviations

| Abbreviation | Description |
|---|---|
| AI | Artificial Intelligence |
| AIOps | Artificial Intelligence for IT Operations |
| ANN | Approximate Nearest Neighbour |
| API | Application Programming Interface |
| AWS | Amazon Web Services |
| CI/CD | Continuous Integration / Continuous Delivery |
| CPU | Central Processing Unit |
| DevEx | Developer Experience |
| DORA | DevOps Research and Assessment |
| DSR | Design Science Research |
| DSV | Department of Computer and Systems Sciences |
| DX | Developer Experience (DX Core 4) |
| GDPR | General Data Protection Regulation |
| GPU | Graphics Processing Unit |
| HNSW | Hierarchical Navigable Small World |
| HyDE | Hypothetical Document Embeddings |
| IDP | Internal Developer Platform |
| IIDP | Intelligent Internal Developer Platform |
| LLM | Large Language Model |
| MCP | Model Context Protocol |
| ML | Machine Learning |
| MTTR | Mean Time to Recovery |
| PR | Pull Request |
| RAG | Retrieval-Augmented Generation |
| RDF | Resource Description Framework |
| REST | Representational State Transfer |
| SPARQL | SPARQL Protocol and RDF Query Language |

# List of Figures

2.1 Conceptual framework: the derivation from theory to evaluation. The feedback arrow reflects a central finding; the durable principles are separable from their current realisation in the artifact. . . . . 13

4.1 RootOps logical architecture. The ingestion layer converts heterogeneous data into a homogeneous internal representation. The semantic memory layer holds embeddings, LogConcepts, and the knowledge graph. The retrieval and synthesis layer runs query planning, three-layer retrieval, reranking, and synthesis. The interaction layer exposes the system through a Next.js dashboard and REST API. . . . 25
4.2 RootOps retrieval pipeline from query planning through HyDE expansion, vector, concept, and graph retrieval, reranking, provenance assembly, and grounded synthesis. . . . . 26
4.3 Default RootOps deployment topology using Docker Compose with API, dashboard, PostgreSQL/pgvector, and configurable local or cloud language-model backends. . . . . 27

# List of Tables


3.1 DSR activities and their realisation in this research . . . . . 16
3.2 Evaluation matrix: criteria, sub-question, and data source . . . . . 19

4.1 Design requirements derived from the problem analysis . . . . . 24
4.2 Core capabilities of RootOps and their mapping to design requirements 27
4.3 Technology stack across RootOps components . . . . . 30

5.1 Participants and recordings . . . . . 35
5.2 Theme support across participants. The vendor participant (P08) is listed separately, as their contributions are reported as a market-level view where they inform a theme. . . . . 36

6.1 Validity threats, corresponding trustworthiness criteria, and mitigations . . . . . 49

D.1 Top-level structure of the RootOps repository . . . . . 67
D.2 Software tools used in the research . . . . . 68

# Contents

**Abstract** **iii**

**Acknowledgements** **v**

**Abbreviations** **vii**

**1 Introduction** **1**
- 1.1 Research Problem . . . 2
- 1.2 Research Question . . . 4
- 1.3 Purpose and Scope . . . 4
- 1.4 Delimitations . . . 5
- 1.5 Expected Contributions . . . 5
- 1.6 Thesis Structure . . . 6

**2 Extended Background** **7**
- 2.1 Internal Developer Platforms . . . 7
- 2.2 DevOps and Software Delivery Performance . . . 8
- 2.3 AIOps and AI in Software Operations . . . 9
- 2.4 Retrieval-Augmented Generation and Semantic Search . . . 10
- 2.5 Synthesis and Gap . . . 11
- 2.6 Conceptual Framework . . . 12

**3 Method** **15**
- 3.1 Research Strategy: Design Science Research . . . 15
- 3.2 Data Collection . . . 17
- 3.3 Data Analysis . . . 19
- 3.4 Ethical Considerations . . . 20
- 3.5 Quality Criteria . . . 21

**4 The RootOps Artifact** **23**
- 4.1 Overview and Design Rationale . . . 23
- 4.2 System Architecture . . . 24

4.3 Core Capabilities . . . 27
4.3.1 Capability 1: Semantic Code Retrieval . . . 27
4.3.2 Capability 2: Cross-Modal Incident Correlation . . . 28
4.3.3 Capability 3: Semantic Pull-Request Risk Scoring . . . 28
4.3.4 Capability 4: Auto-Heal Diagnosis and Remediation . . . 28
4.3.5 Capability 5: Developer Residency Profiling . . . 28
4.4 Key Design Decisions . . . 28
4.4.1 Decision 1: A Single Embedding Space, Augmented by a Graph 29
4.4.2 Decision 2: Retrieval-Grounded Synthesis with Bounded Memory . . . 29
4.4.3 Decision 3: Provenance as a Structural Property, Not a Feature 29
4.4.4 Decision 4: Local-First Deployment with Pluggable Cloud Backends . . . 29
4.5 Implementation Summary . . . 30
4.5.1 Technology Stack . . . 30
4.5.2 Repository Structure . . . 30
4.5.3 The Demonstration Target . . . 30
4.5.4 Demonstration Deployment Configuration . . . 31
4.5.5 Production-Readiness Boundary . . . 31
4.6 Mapping the Artifact to IIDP Principles . . . 31

**5 Results** **33**
5.1 Introduction . . . 33
5.2 Participant Overview . . . 34
5.3 Findings on Architectural Principles (SQ1) . . . 34
5.3.1 Theme 1: Epistemic Stress Is Recognised in Practitioners' Own Terms . . . 34
5.3.2 Theme 2: The IIDP Pattern Is Emerging Independently in Industry . . . 35
5.3.3 Theme 3: Vector-Based Retrieval-Augmented Generation Is Contested . . . 37
5.3.4 Theme 4: Boundary and Scope as an Unresolved Architectural Question . . . 37
5.3.5 Theme 5: Cross-Functional Knowledge as IIDP Affordance . 38
5.4 Findings on Impact (SQ2) . . . 39
5.4.1 Theme 6: Fragmentation as Lived Experience . . . 39
5.4.2 Theme 7: Trust Is Graded by Consequence and by Data Residency . . . 39
5.4.3 Theme 8: Adoption Is Gated by Integration Breadth and Friction 40

5.4.4 Theme 9: Future Trajectory: Skill Convergence, Role Shift, Agent Substrate . . . 41
5.4.5 Theme 10: Developer Profiling Carries an Ethical Surface (single-source) . . . 42

**6 Discussion** **43**
6.1 The Retrieval Substrate Question . . . 43
6.2 Boundary and Scope as an Open Architectural Problem . . . 44
6.3 Governance, Ethics, and the Developer-Profiling Surface . . . 45
6.4 From Predictive Ambition to Proactive, Continuous Intelligence . . . 46
6.5 Critical Reflection on the Artifact . . . 47
6.6 Ethical and Societal Implications . . . 47
6.7 Limitations . . . 48

**7 Conclusions** **51**
7.1 Answering the Research Questions . . . 51
7.2 Contributions . . . 52
7.3 Implications and Future Work . . . 53
7.4 Closing Reflection . . . 53

**Bibliography** **55**

**Appendices** **59**

**A Consent Form** **59**

**B Interview Questions** **61**

**C Supplementary Resources** **65**
C.1 Source Code Repositories . . . 65
C.2 Demonstration Target Deployment . . . 65
C.3 Supporting Material . . . 65

**D Implementation and Tooling Details** **67**
D.1 RootOps Repository Structure . . . 67
D.2 Software Tools . . . 67

# 1

# Introduction

The increasing complexity of modern software systems, characterized by cloud deployments, advanced microservices, distributed systems, and, more recently, near-universal adoption of AI, has driven organizations toward Internal Developer Platforms (IDPs) to standardize and streamline software delivery [1]. This architectural complexity in modern software environments results in a proliferation of tools, data silos, and manual configurations, which drastically increase the cognitive load on development teams [2]. Fragmented intelligence and a reactive posture often characterize the supporting operational landscape, making it difficult to achieve consistent, high-velocity software delivery [2, 3]. IDPs abstract away this operational complexity and reduce developers' cognitive load by providing self-service capabilities for infrastructure provisioning, deployment automation, and observability [4]. Industry research shows that organizations with mature platform engineering practices report significantly faster time-to-market and improved developer satisfaction [5, 6].

The DevOps movement, which emerged in the late 2000s, changed the way organizations think about software delivery. By breaking down silos between development and operations teams, DevOps practices enable organizations to deploy software more frequently and reliably [7]. The DevOps Research and Assessment (DORA) research program has established clear empirical links between technical practices and organizational performance through four now-standard delivery metrics, detailed in Section 2.2 [8].

However, despite two decades of DevOps adoption, many organizations still struggle to achieve elite performance levels, and most remain far from the benchmarks high-performing teams reach [9]. This persistent gap suggests that automation alone may be insufficient, and that intelligent decision support may be needed to help teams navigate the complexity of modern software systems. These metrics are used here to characterise the delivery-performance problem, not as the yardstick of this study's evaluation; the artifact is assessed through demonstration and expert interview rather than longitudinal measurement (Section 1.4).

The emergence of Artificial Intelligence (AI) and Machine Learning (ML) capabilities presents new opportunities to address this gap. AIOps, the application of AI to IT operations, has gained significant attention for automating incident detection, root-cause analysis, and remediation [10]. Recently, large language models (LLMs) have shown remarkable capabilities in code understanding, generation, and explanation [11]. However, these capabilities remain disconnected from the core platform infrastructure and are deployed as point solutions rather than as foundational components of the delivery pipeline.

This thesis investigates the design and implementation of an AI-native Internal Developer Platform, one where AI/ML capabilities are embedded as a foundational layer rather than added as an afterthought. The author's professional experience as a DevOps Engineer in a platform team motivates this study; firsthand observations of platform limitations informed the design of an alternative approach.

## 1.1 Research Problem

Despite growing scholarly attention to Internal Developer Platforms, AIOps, and retrieval-augmented generation, three persistent problems at their intersection remain insufficiently addressed in both practice and research.

**First, fragmented intelligence across the software lifecycle remains an unsolved problem in platform research.** Tools for version control, CI/CD, logging, monitoring, and incident management operate independently, each generating vast amounts of data that rarely cross tool boundaries [12, 2]. A developer investigating a production incident must manually correlate information from Git commits, deployment logs, application traces, and monitoring dashboards, increasing cognitive load and extending mean time to resolution [10]. Harvey et al. [13] explicitly note that current IDPs, while effective at organizing and orchestrating existing assets, lack the capability to comprehend the semantic meaning and operational context of interconnected components. This limitation has been identified as a central driver of the shift toward AI-native software engineering [14],

yet no systematic design research has proposed or evaluated an architecture that unifies these disparate data streams into a single queryable semantic space.

**Second, the prevailing reactive posture in software operations reflects a gap in continuous, system-specific intelligence.** Notaro et al. [3], in their systematic mapping of AIOps literature, observe that most contemporary research disproportionately emphasizes isolated log analysis and anomaly detection at the expense of holistic prevention. They advocate for a structural shift that integrates AIOps directly into core development workflows rather than treating operations as an isolated downstream concern. Despite this call, the literature lacks empirical studies that design and evaluate platforms embedding continuous, system-specific intelligence within the developer workflow itself.

**Third, existing AIOps approaches rely overwhelmingly on generic, general-purpose models** that lack understanding of a specific system's unique patterns, history, and failure modes [15, 16]. An AI model trained on generalized log patterns cannot recognize that a specific type of code change previously caused a particular error in a specific system. While retrieval-augmented generation (RAG) has been demonstrated as effective for grounding LLM outputs in external knowledge [17, 18], it has not been systematically applied to create a unified semantic memory over both source code and operational telemetry. This represents a missed opportunity to develop context-specific, system-aware intelligence.

These three problems may point to a deeper issue. This study interprets the difficulty in modern software delivery as less a shortage of code or tooling than a shortage of understanding: engineering teams appear to produce code faster than they can comprehend it. Code resides in Git, bugs in issue trackers, logs in observability tools, and architectural knowledge in the minds of senior engineers. The recent acceleration of AI-assisted code generation plausibly widens this gap further; tools that raise the rate of code production do not, by themselves, raise the rate at which that code is understood [19]. To name this condition, the study introduces *epistemic stress* as a conceptual lens: the situation in which systems are operated without adequate institutional understanding of their behaviour and implications. The term is an analytical device rather than a measured construct; its grounding in knowledge management theory is developed in Section 2.6, and in the remainder of this chapter the condition is referred to in plain terms as the gap between code produced and code understood.

Other explanations for operational inefficiency exist (governance failures, technical debt, and insufficient training among them [8]), but these do not account for the architectural limitations of the platforms themselves. This thesis takes the position that these limitations are architectural rather than incidental; and, to the best of the author's knowledge, no study has systematically designed and evaluated

a platform architecture that addresses the resulting fragmentation through unified, AI-native intelligence. This is the research gap the thesis addresses.

## 1.2 Research Question

Based on the identified research problem, this study addresses the following research question:

> **RQ:** *How can an AI-native Internal Developer Platform be designed to build unified, context-specific intelligence from software lifecycle data, and what are its effects on software delivery, developer experience, and governance?*

This overarching question is decomposed into two sub-questions:

**SQ1:** What architectural principles and mechanisms are required for an Intelligent Internal Developer Platform to build unified, context-specific intelligence from heterogeneous lifecycle data?

SQ1 is primarily addressed through the design and development of the artifact.

**SQ2:** What is the impact of such a platform on software delivery, developer experience, and governance?

SQ2 is addressed through demonstration-based expert evaluation: structured demonstrations of the artifact followed by semi-structured interviews with practitioners.

## 1.3 Purpose and Scope

This study designs, implements, and evaluates an AI-native Internal Developer Platform component that demonstrates how unified semantic intelligence can support software delivery, and evaluates that proposition through expert judgement rather than measured delivery metrics. The research follows the Design Science Research method, producing both a practical artifact and generalizable knowledge about AI-native platform design.

The scope of this research includes:

- Design and implementation of RootOps, a working platform that ingests code and logs into a unified semantic memory
- Demonstration of key capabilities, including semantic code search, PR risk scoring, auto-heal diagnosis, and developer profiling

- Demonstration-based evaluation with practitioners, gathering expert feedback through semi-structured interviews
- Synthesis of design principles and adoption considerations for AI-native platform work, derived from the expert evaluation

## 1.4 Delimitations

First, the study demonstrates core AI-native platform capabilities rather than a production-hardened enterprise platform. RootOps validates the unified semantic memory approach, but it is not built for high availability, multi-tenancy, or enterprise-scale operations.

Second, evaluation is based on structured demonstrations against a real, public application codebase rather than longitudinal deployment inside participants' organisations. The study therefore validates feasibility and architectural principles, not sustained organisational transformation or comparative performance against commercial tools.

Third, the artifact focuses on code and log data as the primary telemetry sources. The architecture can accommodate traces, metrics, and design documents, but the implementation and evaluation use code and logs as the proof of concept for cross-modal semantic correlation.

## 1.5 Expected Contributions

A framing note applies to all three contributions. The principal contribution of this thesis is the *articulation and structuring* of the Intelligent Internal Developer Platform pattern; a named, teachable formulation of an architectural shape that, as the evaluation in Chapter 5 indicates, practitioners in industry are already converging on independently. The thesis does not claim to have invented the pattern, and the empirical material is read as evidence of its emergence rather than of its novelty. The contributions below should be understood in that light.

The **artifact contribution** is RootOps, a working AI-native Internal Developer Platform prototype that unifies code and log context in a shared semantic memory.

The **empirical contribution** is an expert-practitioner evaluation of the artifact's value, credibility, adoption implications, and governance surface.

The **practical contribution** is a set of design principles and implementation lessons for organisations exploring AI-native platform capabilities.

## 1.6 Thesis Structure

The remainder of the thesis proceeds from grounding to evaluation: Chapter 2 reviews the literature and states the research gap, Chapter 3 sets out the Design Science Research method and evaluation protocol, and Chapter 4 presents the RootOps artifact. Chapter 5 reports the ten interview themes against the two sub-questions, Chapter 6 interprets them against the design claims and states limitations, and Chapter 7 answers the research questions and outlines future work.

# 2

# Extended Background

This chapter provides the scientific grounding for the study, reviewing literature across four domains: Internal Developer Platforms, DevOps and software delivery performance, AIOps and AI in operations, and retrieval-augmented generation with semantic search. The chapter concludes by identifying the specific research gap that this thesis addresses.

## 2.1 Internal Developer Platforms

Contemporary software development is shaped by cloud-native architectures, microservices, and continuous delivery practices that collectively introduce significant operational complexity into the engineering ecosystem [8, 13]. To mitigate this complexity, Internal Developer Platforms (IDPs) have emerged as a strategic necessity. An IDP functions as a unified, curated layer above cloud and infrastructure components, exposing self-service tools, services, and workflows that empower development teams to build, deploy, and operate applications without deep specialised expertise in foundational infrastructure [6, 20].

The foundation for IDPs is rooted in platform engineering, where Skelton and Pais [4] formally delineated platform teams as a distinct organisational topology essential for scaling modern software delivery. Their framing positions the internal platform as a genuine product where developers are explicitly treated as primary customers [21], ensuring that the IDP is engineered with the same rigorous focus on usability, reliability, and iteration that any external commercial product receives.

Industry analyses have converged on a set of essential IDP components: a centralised service catalogue acting as a single source of truth for services and

infrastructure, robust automation for consistent provisioning, standardised deployment pipelines, observability integrations for monitoring and tracing, and embedded security controls that enforce compliance from the earliest stages of the lifecycle [22, 23]. Industry uptake is reflected in the rapid adoption of platforms such as Spotify's Backstage and commercial offerings such as Port, Cortex, and Humanitec.

Despite these benefits, current IDPs share a fundamental architectural limitation: they excel at organising, presenting, and orchestrating existing assets, but cannot comprehend the meaning or operational context of those assets [13]. The gap is easiest to see by example. A conventional service catalogue records that a microservice exists, who owns it, and where its logs are kept. It cannot answer the questions engineers actually ask: what the service does for the end customer, how its API contract has evolved, or which downstream consumers a database-schema change would break. Organising facts is not the same as extracting meaning or reasoning about consequences, and it is this gap that motivates the shift toward AI-native software engineering and agentic operations [14].

A related limitation concerns governance within the software delivery lifecycle. In this study, governance refers specifically to the continuous verification of compliance, the automated enforcement of delivery policies, and the maintenance of auditable decision trails across the software supply chain. Current IDPs typically implement governance through checkpoint-based gates; manual approvals, static policy rules, and periodic audits; rather than as a continuous property of the platform [8, 23]. This reactive approach creates blind spots between checkpoints and scales poorly as delivery velocity rises. An AI-native platform architecture has the potential to transform governance from a discrete checkpoint activity into a continuous, data-driven process embedded within the delivery workflow itself.

## 2.2 DevOps and Software Delivery Performance

DevOps emerged as a response to the systemic friction generated by the historical siloing of development and operations teams [24, 7]. Initially conceptualised in 2009, the movement has matured into a synthesis of cultural philosophies, technical practices, and automated tooling designed to compress the delivery lifecycle and enhance organisational agility [25]. By dismantling rigid organisational barriers, DevOps frameworks foster continuous integration and continuous delivery, mitigating the inefficiencies that historically impeded rapid software deployment.

The empirical validation of these practices has been most rigorously articulated through the DORA programme, spearheaded by Forsgren, Humble, and Kim [8]. DORA defines four primary metrics for benchmarking software delivery:

*deployment frequency*, measuring the cadence of production releases; *lead time for changes*, measuring the duration from code commit to production; *change failure rate*, measuring the proportion of releases requiring remediation; and *mean time to recovery (MTTR)*, measuring the speed of service restoration after an incident [8].

Longitudinal DORA analyses show that elite performers consistently achieve multiple daily deployments alongside change lead times and service recovery durations of less than an hour, while maintaining change failure rates below fifteen percent. This empirical record refutes the traditional axiom that delivery velocity intrinsically compromises operational stability: speed and reliability behave as mutually reinforcing outcomes when underpinned by rigorous automation, continuous testing, and collaborative practices [8, 26].

Contemporary DORA research and related academic inquiries have expanded beyond operational telemetry to include reliability outcomes and holistic developer experience. The DX Core 4 framework conceptualises developer experience across four dimensions: the efficiency of feedback loops, the mitigation of extraneous cognitive load, the facilitation of uninterrupted flow states, and overall developer satisfaction [27]. This broadened analytical framing acknowledges that while infrastructural tooling is essential for optimising delivery metrics, sustainable software engineering ultimately requires structural support for the human experience inherent in the development lifecycle.

## 2.3 AIOps and AI in Software Operations

AIOps was formally introduced by Gartner in 2017 as the strategic integration of artificial intelligence into IT operations to mitigate the complexity of modern distributed software ecosystems. The aim is to automate routine operational workflows, reduce alert fatigue, and expedite incident resolution, transitioning organisational postures from reactive troubleshooting to predictive system maintenance [28].

Foundational work [10] delineates the capabilities necessary for mature AIOps: sophisticated anomaly detection, automated root-cause analysis, dynamic incident correlation, and predictive health monitoring. Practical implementations encounter substantial friction, however: the scarcity of accurately labelled training data, the heterogeneity of operational streams across distributed architectures, and the persistent difficulty of building human trust in autonomous, AI-generated operational recommendations [10, 19]. Notaro et al. [3] mapped the field and observed that most contemporary research disproportionately emphasises isolated log analysis and anomaly detection at the expense of holistic prediction and prevention, advocating for a structural shift that integrates AIOps directly into

core development workflows rather than treating operations as an independent, downstream concern.

The proliferation of large language models has catalysed new trajectories for operational intelligence. These models exhibit capabilities across diverse engineering tasks, from autonomous code generation [11] to algorithmic explanation and automated remediation. AI-assisted tooling such as GitHub Copilot has achieved broad industry adoption, although empirical evaluations show mixed effects on overall code quality: while AI amplifies delivery throughput, it can simultaneously elevate change failure rates if foundational testing and validation loops cannot keep pace [29, 19]. Rigorous application of LLMs to complex operational duties, including nuanced log interpretation and automated incident summarisation, remains a highly active research domain [23].

A fundamental limitation of prevailing AIOps methodologies is their overwhelming reliance on generic, general-purpose models. A model trained only on broad, generalised log patterns inherently lacks the capacity to comprehend the specific topological history, unique failure modes, or localised business context of a bespoke enterprise system [15, 16]. The implication is that operational intelligence must be system-specific. A platform should learn from a bounded scope; the organisation's own code, policies, and telemetry; rather than from generic patterns, so that its reasoning reflects the system it serves. This thesis positions that bounded learning as one of the constitutive design principles of the IIDP pattern.

## 2.4 Retrieval-Augmented Generation and Semantic Search

Retrieval-Augmented Generation (RAG) fuses the information retrieval capabilities of traditional search systems with the contextual synthesis of generative language models. Rather than relying only on the static, parametric knowledge encoded during model training, RAG architectures dynamically retrieve relevant documents from an external knowledge base and use them to ground the model's generation [17]. This addresses several fundamental limitations of standalone large language models, most notably their susceptibility to hallucination and reliance on temporally frozen training corpora. By anchoring generated responses in explicitly retrieved evidence, RAG systems yield outputs that are more accurate, verifiable, and contextually up-to-date. The technique has achieved widespread adoption within complex enterprise applications and Intelligent Internal Developer Platforms, where operational accuracy, auditability, and precise traceability are non-negotiable requirements [18, 30].

The foundational mechanism enabling effective RAG is semantic search, operationalised through dense vector embeddings. Advanced embedding models such as Sentence-BERT [31] map unstructured text into high-dimensional dense representations within which semantic similarity can be quantified using metrics such as cosine distance. This empowers the search mechanism to comprehend underlying contextual meaning rather than relying on lexical matching, bridging the semantic gap between user queries and stored organisational knowledge. The same family of techniques has been extended to other modalities, including speech recognition at scale [32].

To support efficient storage and rapid retrieval of high-dimensional embeddings, specialised vector databases; including Pinecone, Weaviate, and the pgvector extension for PostgreSQL; have become indispensable infrastructure components. These systems typically leverage the Hierarchical Navigable Small World (HNSW) algorithm, which optimises approximate nearest-neighbour (ANN) searches to deliver near real-time querying latency even across enterprise-scale datasets. The engineering of such systems is non-trivial, however, and recent work has documented a set of recurring failure points that production RAG deployments must address [33].

This study applies these techniques in a less common configuration: source code and operational logs are embedded into one shared vector space using a single embedding model. The benefit is cross-modal querying. A question about a production bug can retrieve both the relevant code and the correlated log entries, ranked together by relevance. This shared representation is what the thesis calls the substrate: the common layer in which lifecycle data are stored and queried, and on which the IIDP pattern depends.

## 2.5 Synthesis and Gap

The literature reviewed in this chapter is individually mature but collectively adjacent. Each strand addresses one slice of the modern engineering organisation's problem. Platform engineering describes how to build a self-service infrastructure layer. DevOps and its measurement frameworks describe the delivery outcomes such platforms are meant to enable. AIOps describes how machine learning can be applied to a subset of the operational signal. RAG and semantic search describe how generative models can be grounded in external knowledge. What none of them provides, individually or in combination, is a platform whose architecture treats AI as foundational infrastructure rather than as an added capability.

The research gap this thesis addresses is therefore specific. The literature treats the platform and the intelligence as separable concerns: platforms are evaluated

on what engineers can do through them, and intelligence systems are evaluated on the tasks they automate. This thesis proposes, as a design hypothesis to be examined rather than a settled claim, that the next form of the developer platform integrates these concerns by design. That integration rests on four elements: a unified semantic substrate spanning code and operational data; system-specific learning bounded to that substrate; AI as the foundational layer through which the platform is queried and acted upon; and governance expressed as a continuous property of the substrate rather than an external audit function. Chapter 3 sets out the method by which the thesis articulates and evaluates this pattern, and Chapter 4 presents its instantiation.

## 2.6 Conceptual Framework

The preceding sections reviewed four literature domains and identified the gap this thesis addresses. This section makes explicit the conceptual chain that connects that gap to the empirical evaluation, so that the role of each component of the thesis is unambiguous.

The chain has four links. The first is *theory*: the condition this thesis names *epistemic stress*; the gap between the rate at which software is produced and the rate at which it is understood; together with the literature's account of fragmented intelligence, reactive operations, and generic models. Epistemic stress is used in this thesis as an *analytical lens and design motivation*: it is not advanced as a formally measured construct, but as a named condition that motivates the design and against which practitioner recognition can be assessed.

The lens is not coined in a theoretical vacuum; it names a software-delivery configuration of problems long studied across several research traditions, and its relation to each can be stated precisely. Knowledge management distinguishes tacit knowledge, held by individuals, from codified knowledge, held in artefacts, and treats the conversion between them as the central organisational challenge [34, 35]; organisational memory research examines how organisations retain and retrieve what they have learned [36]; sensemaking describes how actors construct plausible understanding of equivocal events, characteristically after the fact [37]; distributed cognition locates cognitive work in systems of people and artefacts rather than in individual minds [38]; cognitive load theory addresses the limits of individual working memory under task demands [39]; and the sociotechnical tradition insists that technical and social subsystems be designed jointly [40]. Each of these describes a facet of the condition this thesis observes in software delivery.

What epistemic stress names is the compound, rate-driven form these facets take in modern software organisations, and this is where it differs from each parent

concept. It is not cognitive load, because it is a property of the organisation rather than of an individual's working memory: it persists across personnel and worsens with tenure gaps, as the empirical material in Chapter 5 illustrates. It is not an organisational-memory deficit alone, because the artefacts are retained; what is missing is the conversion of retained artefacts into usable understanding. It is not sensemaking failure in Weick's episodic sense, because it is chronic and ambient rather than triggered by a disruptive event. Stated in distributed-cognition terms, it is the condition in which the artefactual side of the cognitive system (code, logs, tickets, history) grows faster than the representational tools that let the human side read it. Epistemic stress is therefore offered as a bridging construct: a named, recognisable condition at the intersection of these literatures, specific to settings where automated production outpaces institutional comprehension, used here as an analytical lens and a design motivation rather than as a measured variable.

The second link is the set of four *design principles* derived from that theory: a unified semantic substrate across modalities; system-specific learning over a bounded scope; AI as a foundational layer rather than an addition; and governance as a continuous, embedded property. These principles are the thesis's primary theoretical claim; the articulation of the Intelligent Internal Developer Platform (IIDP) pattern. The third link is the *artifact*, RootOps, which instantiates the four principles in working, reproducible form and demonstrates that the pattern is buildable. The fourth link is *evaluation*: structured demonstration and expert interviews, which test whether practitioners recognise the motivating condition, judge the principles sound, and find the instantiation credible. Figure 2.1 shows the chain. It is read left to right as a derivation; theory motivates principles, principles are instantiated as an artifact, the artifact is evaluated; and the evaluation feeds back onto the principles, since a central finding of this thesis is the separation of the durable principles from their current, contestable realisation.

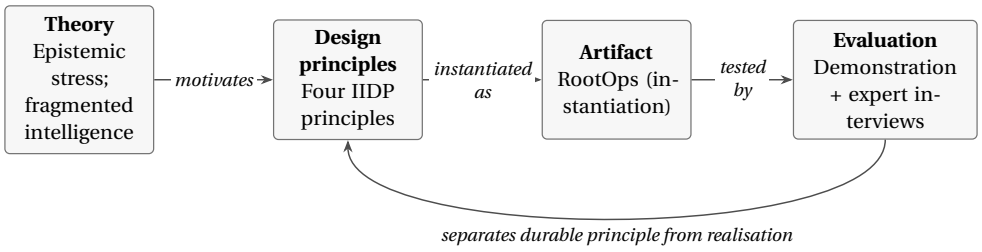



Figure 2.1: Conceptual framework: the derivation from theory to evaluation. The feedback arrow reflects a central finding; the durable principles are separable from their current realisation in the artifact.

# 3

# Method

This chapter describes the research method employed in this study. It presents the research strategy, data collection methods, data analysis methods, ethical considerations, and quality criteria.

## 3.1 Research Strategy: Design Science Research

This study adopts the Design Science Research (DSR) paradigm. DSR is a well-established approach within Information Systems research for studies that involve creating artifacts; constructs, models, methods, or instantiations; intended to solve practical problems [41]. In contrast to behavioural research, which primarily seeks to explain, predict, or observe existing phenomena, DSR is constructive: it focuses on the proactive creation and rigorous evaluation of novel, purposeful artifacts designed to resolve complex, practical problems. The knowledge contribution in DSR comes from both the artifact itself; demonstrating that something is possible; and the evaluation that establishes under what conditions and with what effects.

DSR is appropriate here for four reasons. The research goal is to create a novel artifact, an AI-native platform component, that does not exist in current practice. The research addresses a practical problem, fragmented intelligence in software delivery, that is simultaneously a gap in the scholarly literature. Evaluation through expert assessment generates knowledge beyond the specific implementation. The framework accommodates both a technical contribution (the artifact) and an empirical contribution (the evaluation findings). To ensure rigour, this study follows the seven foundational guidelines articulated by Hevner et al. [41] and structures

its activities according to the five-activity DSR framework proposed by Johannesson and Perjons [42]. Table 3.1 shows how each activity is realised in this research. The activities are not strictly sequential; design science is inherently iterative, and the artifact was refined through multiple cycles of development, demonstration, and informal evaluation before the formal evaluation phase. The strategy-and-evaluation pairing adopted here, a constructed software artifact assessed through structured demonstration and practitioner interview rather than field deployment, follows the pattern of recent design-oriented studies of LLM-integrated and AI-native software systems, which likewise combine a working system contribution with qualitative expert assessment [30, 18, 14], and is an evaluation form explicitly sanctioned in the DSR methodological literature [42, 41].

Table 3.1: DSR activities and their realisation in this research

| **DSR Activity** | **Realisation in this research** |
|---|---|
| Explicate Problem | Literature review identifies fragmented intelligence, reactive posture, and generic models as key gaps in IDP and AIOps research |
| Define Requirements | Requirements derived from problem analysis: unified semantic memory, system-specific learning, AI as the foundational layer, embedded governance |
| Design and Develop | Development of RootOps: hybrid RAG architecture with pgvector, sentence-transformers, and LLM integration |
| Demonstrate | Structured demonstrations to practitioners using a real, publicly deployed business management application as the ingestion target, covering semantic code retrieval, cross-modal correlation, PR risk scoring, auto-heal diagnosis, and developer profiling |
| Evaluate | Demonstration-based evaluation with expert interviews assessing perceived value, trust, governance implications, and adoption factors |

Several alternative strategies were considered before selecting DSR. A case study suits the investigation of existing phenomena, but no AI-native IDP implementations with unified semantic memory were available for study, and the goal here is to create rather than to investigate. Action research would require intervention cycles exceeding a master's-thesis timeline. A quantitative or experimental evaluation was considered and set aside on feasibility grounds rather than preference: measuring the platform's effect on delivery outcomes (for example,

the DORA metrics of Section 2.2) would require longitudinal deployment, a stable baseline, and a control condition, none of which was attainable within the access and time constraints of this study; a short-horizon measurement would risk attributing noise to the artifact. The evaluation is therefore demonstration-based and interpretive, establishing feasibility and practitioner-judged value, with longitudinal quantitative evaluation identified as future work (Chapter 7). DSR accommodates both the technical work of building an artifact and the empirical work of evaluating it, making it the most suitable strategy.

## 3.2 Data Collection

This study employs a demonstration-based evaluation strategy with semi-structured interviews as the primary data collection method. This approach is consistent with DSR, where Johannesson and Perjons [42] identify informed argument and expert evaluation as valid evaluation methods alongside field deployment. Rather than deploying the artifact in a participant's own production environment, the researcher demonstrates RootOps against a real, publicly available application and conducts interviews to capture participants' assessments.

Each evaluation session follows a structured demonstration protocol. The researcher operates RootOps against an ingested copy of the CORE Biz Manager application (`theorjiugovictor/core-v1`), a real, publicly deployed business-management platform with 107 commits across its development history. The application is a Next.js/TypeScript codebase implementing authentication, persistence, an AI-assisted command console, and inventory and sales management features. Its deployed instance and source repository are both publicly accessible, allowing participants to observe the application's surface behaviour alongside the platform's claims about the code that produces it. The demonstration covers five scenarios: semantic code search with natural-language queries, cross-modal correlation between code and logs, semantic pull-request risk scoring grounded in historical bug patterns, auto-heal diagnosis from error logs to generated fix, and developer profiling from commit behaviour. Participants observe the platform and are invited to interact with it throughout the session.

Following each demonstration, a semi-structured interview is conducted with the participant. The interview guide covers five core themes: trust in platform recommendations and explanations, perceived usefulness for daily development and operations work, cognitive load reduction and context-switching, governance implications including accountability and compliance, and adoption barriers and organisational readiness (the full question set is reproduced in Appendix B).

Interviews are conducted via video conference, audio-recorded with consent, and typically last 30–45 minutes.

The final sample comprised eight participants selected through purposive sampling. This sample size is justified on three methodological grounds. First, under the information-power model of Malterud et al. [43], smaller samples suffice when the study aim is narrow, participants hold specificity for the topic, the analysis is supported by established theory, and the researcher-participant dialogue is strong; all four conditions hold here, since every participant is a senior or specialised practitioner assessing a single, concretely demonstrated artifact against a defined evaluation matrix. Second, empirical work on qualitative sample sizes finds that the large majority of themes in expert interview studies surface within the first six to twelve interviews [44, 45]. Third, the evaluation is demonstration-based and depth-seeking rather than survey-based and generalisation-seeking [46]: each session combined a structured demonstration with a 30–45 minute interview, producing dense, directly comparable material per participant rather than broad coverage across many. Participants were selected against three suitability criteria: direct, current experience in software development, DevOps, platform engineering, or an AI-adjacent engineering role; sufficient seniority or specialisation to assess architectural decisions rather than only surface usability; and a spread of organisational contexts, so that recognition of the motivating condition could not be attributed to a single setting. The evaluation itself was structured rather than open-ended. Each session assessed the artifact against five evaluation criteria, derived from the DSR quality guidelines and the research sub-questions: whether practitioners *recognise* the motivating condition; whether they judge the four design principles *sound*; whether they find the instantiation *credible*; how they assess *impact* on delivery, developer experience, and governance; and what *adoption* conditions and barriers they identify. Table 3.2 maps these criteria to the interview themes and the data source for each. Themes were validated through a documented procedure: each candidate theme was required to be supported by an identifiable passage in at least one transcript, was named, defined, and attributed to contributing participants, and every quotation was verified verbatim against the source transcript before inclusion. Themes carried by a single participant were retained only where salience was clear and were flagged as single-source findings.

The software tools used across artifact development, data collection, transcription, anonymisation, analysis, and write-up are listed in Appendix D.

Table 3.2: Evaluation matrix: criteria, sub-question, and data source

| Evaluation criterion | Sub-question | How assessed |
|---|---|---|
| Recognition of epistemic stress | SQ1 | Whether participants describe the motivating condition in their own terms (demonstration reaction; interview section on fragmentation) |
| Soundness of the four principles | SQ1 | Practitioner judgement of the architectural principles, including critique and counter-proposals (principle-by-principle discussion) |
| Credibility of the instantiation | SQ1 / SQ2 | Reaction to the live demonstration against a real deployed application; questions on retrieval, confidence scoring, and architecture |
| Impact on delivery, DevEx, governance | SQ2 | Interview sections on trust, cognitive load, usefulness, and governance implications |
| Adoption conditions and barriers | SQ2 | Interview section on technical, organisational, and cultural barriers to adoption |

## 3.3 Data Analysis

Interview data were analysed following Braun and Clarke's six-phase framework for reflexive thematic analysis [47]. The framework was chosen because it supports the integration of deductive coding; against the four IIDP principles articulated in Chapter 2; with inductive coding, allowing themes to emerge from the data that the deductive framework did not anticipate.

Each of the eight recordings was transcribed locally and then read in full before any coding was attempted. Each transcript was then read with a coding lens applied: codes were assigned to passages on the basis of analytical significance, including passages that validated, contradicted, extended, or added nuance to the four IIDP principles, or that surfaced trust, governance, scope, or adoption material relevant to SQ2. Codes were kept short, typically two to four words, and inductive codes that did not fit the deductive codebook were treated as first-class and flagged for cross-cutting attention. Codes were then clustered into candidate themes on the basis of analytic affinity rather than surface vocabulary, and reviewed against the underlying transcripts to confirm data support. Themes supported by a single participant were retained where they carried clear salience but flagged

as single-source findings warranting analytical caution. Each theme received a noun-phrase name, a definition of two to four sentences, a list of contributing participants, and an explicit mapping to one or both sub-questions. Themes form the structural spine of Chapter 5; within each theme, supporting quotations are reported verbatim with light editorial cleaning of repeated discourse markers, and each quotation was verified against the original transcript. The analysis produced ten cross-cutting themes from the participant-level codes developed across the eight recordings.

A reflexive caution applies to the analysis as a whole. The researcher is simultaneously the designer of the artifact, the operator of each demonstration, and the sole coder of the resulting transcripts; this concentration of roles is a bias surface, since the analyst has a stake in the artifact being judged favourably. The analysis does not claim to eliminate this bias. It constrains it through the procedures above (verbatim quotation verification, single-source flagging, retention of contradicting and critical material as first-class codes) and through the mitigations described in Section 3.5. The prominence given in Chapters 5 and 6 to themes that challenge the design, in particular the retrieval-substrate critique and the unresolved boundary question, is itself a check the reader can apply: an analysis captured by its author's stake would be unlikely to foreground its own strongest counter-evidence.

## 3.4 Ethical Considerations

The research adheres to ethical guidelines for studies involving human participants [46]. Participants received clear information about the study's purpose, how their data would be used, and their rights, and written consent was obtained before interviews proceeded. Participant names and identifying details were replaced with pseudonyms or role descriptors, and direct quotes were reviewed to ensure they did not enable identification. The artifact and the demonstration target are both publicly released open-source repositories authored by the researcher (`github.com/Intelligent-IDP/rootops` and `github.com/theorjiugovictor/core-v1`); no third-party proprietary code, customer data, or organisational internal systems were used. Participation was voluntary, with the right to decline specific questions or withdraw at any time. This research utilised AI assistance in bounded ways: AI tools supported brainstorming, language refinement, and structuring for clarity; AI-assisted coding support was used for generating, debugging, and analysing parts of the RootOps artifact; and Prism was used for LaTeX editing, formatting, diagnostics, and compilation support. The thematic analysis itself; coding, theme development, and verification against transcripts; was conducted manually by the researcher. The use of AI tools in the

thesis process is documented and disclosed; the author remains responsible for all content.

## 3.5 Quality Criteria

The study applies quality criteria at two levels. For the DSR contribution, the seven Hevner et al. [41] guidelines serve as primary criteria, requiring a viable artifact, a relevant problem, rigorous evaluation, a clear research contribution, rigorous methods in construction and evaluation, design as a search process, and effective communication. These guidelines are addressed across the artifact and evaluation chapters.

For the qualitative component, the trustworthiness criteria proposed by Lincoln and Guba [48] are applied. Concerns conventionally expressed as validity, reliability, and researcher bias are addressed here through their established interpretive equivalents: credibility and confirmability in place of internal validity, transferability in place of external validity, and dependability in place of reliability, with researcher bias treated under confirmability and the reflexivity measures described in Section 3.3. Credibility is strengthened by grounding every analytical claim in the interview transcripts and by triangulating participant accounts against the live behaviour of the artifact observed during each demonstration session; transferability is supported by thick description of participant roles, professional backgrounds, and the specific platform capabilities evaluated; dependability is achieved through a documented audit trail of protocols, coding procedures, and analytical decisions, supported by the use of established thematic analysis procedures [47]; and confirmability is addressed by grounding all interpretations in the interview data and maintaining reflexive awareness of the researcher's dual role as designer of the artifact and evaluator of its reception. Three mitigations attach to this dual role: the artifact is released under an open-source licence and operated publicly, so readers may inspect the system rather than rely on researcher narration; during demonstration sessions the researcher operates the platform but the interpretive judgements about value, trust, and adoption are drawn from participant accounts rather than the researcher's own assessment; and illustrative direct quotations accompany analytical claims in Chapter 5, allowing readers to evaluate the strength of the evidence. Transparency is maintained throughout the process by documenting all methodological decisions; all data-collection instruments, interview guides, anonymisation procedures, and analysis records are retained to enable scrutiny and potential replication.

# 4

# The RootOps Artifact

This chapter presents RootOps, the artifact developed in this study. In Design Science Research terms, the chapter documents the *Design and Develop* and *Demonstrate* activities of the Johannesson and Perjons framework [42], building on the problem explication and requirements derivation set out in Chapters 2 and 3. Each architectural choice, capability, and implementation decision is traceable to a requirement that emerged from the literature gap and the problem framing of fragmented intelligence and epistemic stress.

## 4.1 Overview and Design Rationale

RootOps is the proposed response. It is an AI-native Internal Developer Platform that ingests lifecycle-wide telemetry; source code, commit history, runtime logs, and the relationships between them; into a single semantic memory, and exposes that memory through query, review, correlation, and remediation workflows embedded directly in the development flow. Rather than treating artificial intelligence as a feature added on top of an existing platform, RootOps treats AI as the foundational layer of the platform itself. Semantic memory, not a service catalogue, is the organising abstraction. The artifact is a working open-source monorepo released under the Apache 2.0 licence at `github.com/Intelligent-IDP/rootops`. It performs real ingestion against real Git repositories, semantic retrieval against PostgreSQL with pgvector, and synthesis through a configurable language-model backend. The default deployment is Docker Compose and can run entirely on a developer's own machine, with no code, log, or query data leaving the local environment. Following Hevner's guidance that a design science artifact should be

demonstrably functional in a relevant environment [41], RootOps was operated end-to-end against a realistic codebase during expert evaluation sessions rather than presented as mock-ups.

Four requirements emerged from the literature review and problem analysis, summarised in Table 4.1. **R1 Unified semantic memory:** the platform must represent code, logs, commit history, and relationships within a single intelligence so that behavioural symptoms can be traced to source and history in one retrieval flow. **R2 System-specific intelligence:** the platform must learn from a bounded system rather than rely on generic pretrained assumptions. **R3 AI as the foundational layer:** the platform must be built so that semantic memory, retrieval, and synthesis are the mechanism through which it is queried and operated, rather than treating AI as a feature added on top of a conventional catalogue-and-workflow platform. **R4 Embedded continuous governance:** the platform must treat governance as part of the development flow through provenance, data residency, and human accountability for consequential actions.

Table 4.1: Design requirements derived from the problem analysis

| **Requirement** | **Problem addressed** | **Realisation in RootOps** |
|---|---|---|
| R1: Unified semantic memory | Context scatter across code, logs, history, relationships | Shared 768-dimensional embedding space; pgvector persistence; LogConcepts; knowledge graph |
| R2: System-specific intelligence | Generic AIOps models without system knowledge | Per-repository ingestion scope; bounded profiles, concepts, and graph edges |
| R3: AI as the foundational layer | AI treated as a bolted-on feature rather than the platform's organising mechanism | Every capability depends on semantic memory, retrieval, and synthesis; no conventional catalogue function exists with AI disabled |
| R4: Embedded continuous governance | Checkpoint governance incompatible with continuous delivery | Provenance-by-default; semantic PR risk scoring; human-in-the-loop remediation; local-first residency |

## 4.2 System Architecture

RootOps is structured as four logical layers, illustrated in Figure 4.1: an ingestion layer, a semantic memory layer, a retrieval and synthesis layer, and an interaction layer. The separation isolates the stable abstraction, the unified semantic memory,

from replaceable components such as language models, ingestion connectors, and the user-facing surface. The paragraphs below describe ingestion, memory, and retrieval in turn; the figure shows how the layers compose.

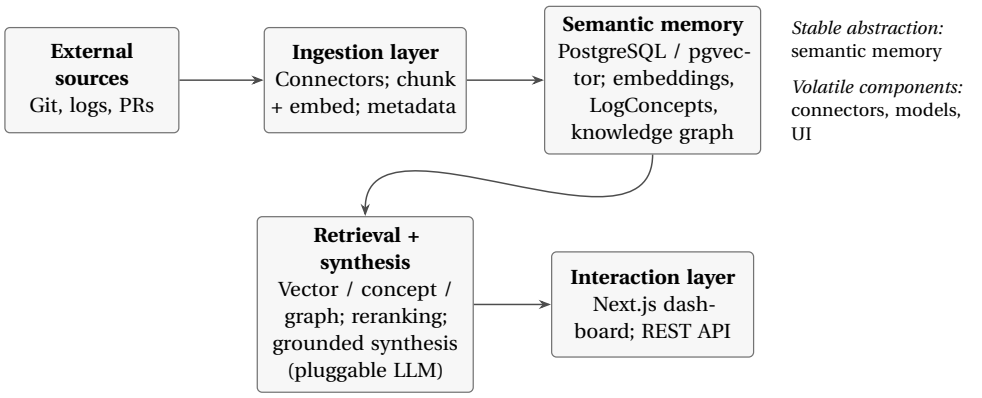



Figure 4.1: RootOps logical architecture. The ingestion layer converts heterogeneous data into a homogeneous internal representation. The semantic memory layer holds embeddings, LogConcepts, and the knowledge graph. The retrieval and synthesis layer runs query planning, three-layer retrieval, reranking, and synthesis. The interaction layer exposes the system through a Next.js dashboard and REST API.

Ingestion converts heterogeneous data into chunks with embeddings and metadata plus relational and conceptual structures derived from them. For code, a Git repository is cloned and divided into chunks at semantic boundaries where possible. Each chunk is embedded with `jinaai/jina-embeddings-v2-base-code` into a 768-dimensional vector, annotated with metadata (file, line, commit, repository, service, author, language, timestamp), and persisted to PostgreSQL alongside an index of commit-level diffs. The 8192-token context window allows entire functions or short files to be embedded as coherent units rather than arbitrary slices. Logs share two invariants with code: the same embedding model is used and the same storage substrate holds the result. Logs arrive through an OpenTelemetry-compatible endpoint or through raw text ingestion in the dashboard, are annotated and embedded, and pass through Drain-style template clustering to derive recurring LogConcepts. Each LogConcept includes a temporal histogram, enabling the platform to surface rising concepts whose frequency is accelerating relative to baseline.

The semantic memory is the conceptual centre of RootOps. Code and logs share one 768-dimensional embedding space, so the cosine distance between a natural-language query and a code chunk is comparable to the distance between the same query and a log line. Two structures extend the memory. LogConcepts represent recurring log templates with temporal trajectories, allowing queries to

operate over event types rather than isolated lines. The knowledge graph represents services, files, errors, commits, and contributors as nodes, with edges for dependencies, authorship, provenance, and causal relations. Causal edges are promoted through confidence levels as evidence accumulates: `observed`, `correlates_with`, `probable_cause`, `confirmed_cause`.

Queries are answered through a multi-stage retrieval-augmented generation pipeline [17]. The query planner classifies intent as diagnostic, architectural, impact, or general. HyDE expansion can generate a hypothetical answer and embed that text to improve retrieval. Three retrievals then run in parallel: vector retrieval over the unified chunks table, concept retrieval over LogConcepts, and graph retrieval over related entities and causal edges. Results are cross-correlated, passed through a cross-encoder reranker, assembled with provenance metadata and a codebase summary, and synthesised by the configured language-model backend using bounded conversation memory.

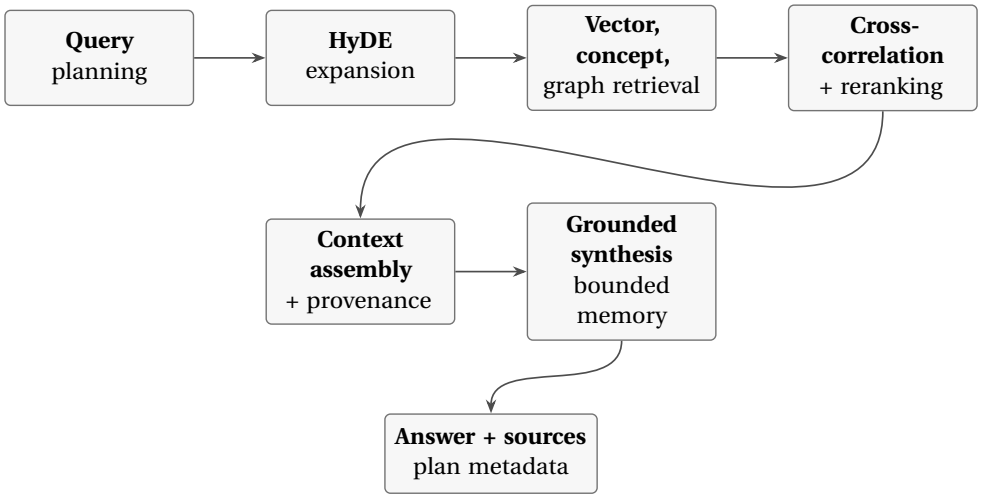



Figure 4.2: RootOps retrieval pipeline from query planning through HyDE expansion, vector, concept, and graph retrieval, reranking, provenance assembly, and grounded synthesis.

RootOps is deployed by default through Docker Compose. The API, web dashboard, PostgreSQL with pgvector, and local language-model runtime run side by side on a single host. The default language-model backend is Ollama for sufficiently provisioned machines; OpenAI, Anthropic, AWS Bedrock, and Google Gemini can be selected through configuration. The platform supports RAM-aware tiering: high-memory hosts run the full local configuration, while constrained hosts disable costly features such as HyDE and codebase-summary generation or use a cloud synthesis backend.

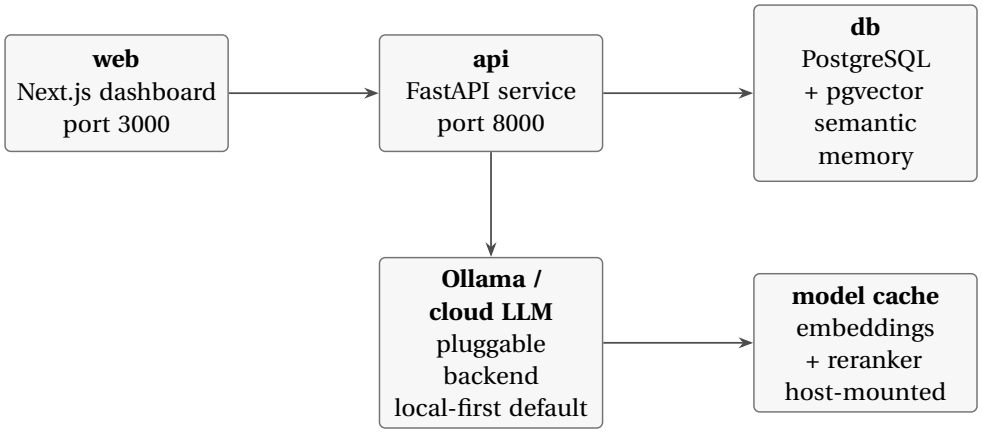



Figure 4.3: Default RootOps deployment topology using Docker Compose with API, dashboard, PostgreSQL/pgvector, and configurable local or cloud language-model backends.

## 4.3 Core Capabilities

The architecture supports five capabilities, summarised in Table 4.2, that together constitute the platform's offering to users and form the units of evaluation in this study; expert participants were guided through scenarios exercising each capability against CORE Biz Manager.

Table 4.2: Core capabilities of RootOps and their mapping to design requirements

| # | Capability | Epistemic problem addressed | Reqs. |
|---|---|---|---|
| 1 | Semantic code retrieval | Code searchable only by keyword, not meaning | R1, R2 |
| 2 | Cross-modal incident correlation | Symptoms in logs cannot be traced to code causes | R1, R2, R3 |
| 3 | Semantic pull-request risk scoring | Reviews lack memory of historical bug patterns | R2, R3, R4 |
| 4 | Auto-heal diagnosis and remediation | Incidents repeatedly diagnosed from scratch | R3, R4 |
| 5 | Developer residency profiling | Institutional knowledge tied to individuals | R2, R4 |

### 4.3.1 Capability 1: Semantic Code Retrieval

Semantic code retrieval allows developers to query by meaning rather than lexical surface. A user asks a natural-language question and receives a synthesised answer

with source chunks, file paths, line ranges, commits, authors, contributing graph evidence, and retrieval-plan metadata. In the demonstration, RootOps answered questions about authentication, Firebase persistence, AWS Bedrock integration, rate limiting, and server actions in the CORE Biz Manager codebase.

### 4.3.2 Capability 2: Cross-Modal Incident Correlation

Cross-modal incident correlation collapses the distance between a runtime symptom and responsible source. Diagnostic queries weight LogConcept and graph retrieval more heavily, then boost code chunks linked to matching logs or graph entities.

### 4.3.3 Capability 3: Semantic Pull-Request Risk Scoring

Semantic pull-request risk scoring brings institutional bug memory into review. A diff is decomposed, embedded, compared against historical code and commits, and scored according to resemblance to bug fixes, sensitive code paths, and graph-linked incident areas. The output is a score plus evidence, not an opaque verdict.

### 4.3.4 Capability 4: Auto-Heal Diagnosis and Remediation

Auto-heal reduces the latency from production error to candidate fix. The platform selects an error or rising LogConcept, retrieves likely source causes, declines if similarity is below threshold, asks the model for a diagnosis and proposed change, stores a pending fix, and only opens a pull request after explicit human approval. This preserves developer agency while automating triage and hypothesis formation.

### 4.3.5 Capability 5: Developer Residency Profiling

Developer residency profiling externalises a subset of institutional knowledge from commit history. For each author, RootOps aggregates languages, files touched, message conventions, and modification patterns, then generates an evidence-grounded contribution fingerprint linked into the knowledge graph.

## 4.4 Key Design Decisions

The architecture and capabilities result from four consequential decisions.

### 4.4.1 Decision 1: A Single Embedding Space, Augmented by a Graph

RootOps embeds code chunks and logs with the same model into one 768-dimensional vector table, while a knowledge graph captures relational and causal context. The alternative was a federated architecture with separate indexes and a separate graph store. The chosen design sacrifices some per-modality optimisation in exchange for first-class cross-modal and relational retrieval.

### 4.4.2 Decision 2: Retrieval-Grounded Synthesis with Bounded Memory

The model receives fresh retrieval results on each query, plus bounded conversation history for follow-up dialogue. The alternative was an unbounded agentic assistant with persistent working memory. The chosen design preserves useful dialogue while limiting hallucination compounding and keeping answers reconstructible from retrieved evidence.

### 4.4.3 Decision 3: Provenance as a Structural Property, Not a Feature

Every answer surfaces provenance; the record of which sources and retrieval steps produced a given answer; through the chunks, LogConcepts, graph edges, and retrieval plan that produced it. The alternative was answer-first generation with related sources attached afterwards. RootOps instead produces answers from provenance-tagged context and exposes the retrieval plan, making unsupported claims and poor retrieval choices more visible.

### 4.4.4 Decision 4: Local-First Deployment with Pluggable Cloud Backends

The default deployment is local Docker Compose, with cloud LLMs optional. The alternative was a managed cloud service. Local-first deployment increases operational responsibility for the adopting organisation, but makes data residency a property of the default architecture rather than a vendor assertion. The four decisions reinforce one another: the shared embedding space makes provenance tractable, retrieval-grounded synthesis makes it reliable, and local-first deployment makes the artifact credible for organisations that cannot export proprietary code or logs.

## 4.5 Implementation Summary

### 4.5.1 Technology Stack

Table 4.3: Technology stack across RootOps components

| Component | Layer | Technology |
|---|---|---|
| `api/` | API framework | FastAPI |
| | Persistence | SQLAlchemy, Alembic, PostgreSQL 15, pgvector |
| | Models | Jina code embeddings; MiniLM cross-encoder reranker; Drain-style log clustering |
| | LLM backends | Ollama, OpenAI, Anthropic, AWS Bedrock, Google Gemini |
| `web/` | UI framework | Next.js 15, React 19, TypeScript 5 |
| | Styling and charts | Tailwind CSS, Recharts |
| Deployment | Orchestration | Docker Compose, GitHub Container Registry images, optional remote Linux host through `make cloud-*` commands |

PostgreSQL with pgvector was selected because embeddings, structured metadata, concepts, graph edges, commits, and profiles can be queried through one connection. The Jina code embedding model was selected for code-aware representations and long context. The cross-encoder reranker improves precision after inexpensive vector retrieval. The pluggable language-model dispatcher accommodates both local-first and organisation-specific cloud deployments.

### 4.5.2 Repository Structure

RootOps is released as a public Apache 2.0 monorepo at `github.com/Intelligent-IDP/rootops`. Its top-level structure is summarised in Appendix D.

### 4.5.3 The Demonstration Target

The artifact was demonstrated against `theorjiugovictor/core-v1`, the CORE Biz Manager application. This is a real Next.js business-management platform with authentication, Firebase Firestore persistence, an AI command console using AWS Bedrock and Claude 3 Haiku, inventory and sales management, business insights, security middleware, rate limiting, and a 107-commit history. Real deployment logs

were ingested, including authentication failures, Firebase quota errors, Bedrock invocation failures, and validation rejections.

### 4.5.4 Demonstration Deployment Configuration

The expert-evaluation deployment used Docker Compose and configured Google Gemini as the synthesis backend through the dispatcher. This choice reflected an 8 GB RAM constraint on the evaluation host: running Ollama alongside the embedding model, reranker, database, and application services was infeasible. This is a documented RAM-tier configuration, not a workaround; hosts with at least 16 GB can run the fully local Ollama configuration.

### 4.5.5 Production-Readiness Boundary

The evaluation deployment is not claimed to be production-ready. Production operation would require management-plane authentication and authorisation, network access controls, backups, horizontal scaling, monitoring, and operational tooling around the database and model runtime. The artifact contribution is the intelligent substrate and workflows, not a complete production operations package. The evaluated baseline is fixed by four identifiers: the RootOps repository commit, the configured language-model backend, the embedding model version, and the CORE Biz Manager commit at ingestion time, held constant across all sessions so that qualitative findings are comparable.

## 4.6 Mapping the Artifact to IIDP Principles

RootOps instantiates all four principles that distinguish an Intelligent Internal Developer Platform from a conventional IDP or a bolted-on AIOps capability. The realisation of each is traceable through the requirements in Table 4.1 and the capability mapping in Table 4.2: a unified intelligence over code, history, telemetry, LogConcepts, and graph relationships (Principle 1); system-specific learning derived from the bounded history of the ingested system, with multi-repository scoping through `repo_id` (Principle 2); and governance expressed through provenance, automatic risk visibility, bounded automation, and local-first residency (Principle 4). **Principle 3 (AI as foundational, not additive)** is the load-bearing case: every capability depends on semantic memory, retrieval, and synthesis, and with semantic memory empty the platform does not fall back to a conventional service-catalogue function; the intelligent substrate is the platform.

The next chapter turns to the empirical material gathered through expert interviews, and what it implies for the proposition that AI should be foundational infrastructure for software delivery rather than a feature added to it.

# 5

# Results

This chapter presents the findings from the eight expert-interview sessions described in Chapter 3, structured to answer the two sub-questions of the study in turn. Section 5.1 introduces the analytical conventions. Section 5.2 characterises the participants. Section 5.3 reports the five themes that speak to SQ1, the architectural principles required for an IIDP. Section 5.4 reports the five themes that speak to SQ2, the platform's impact on software delivery, developer experience, and governance.

## 5.1 Introduction

The thematic analysis followed Braun and Clarke's six-phase framework as adopted in Section 3.3. Codes were clustered by the researcher into ten cross-cutting themes, each defined, attributed to contributing participants, and mapped to the research questions. Every code, theme, and quotation reported in this chapter was developed and verified by the researcher against the original transcripts. Themes are presented in the order of the sub-question they primarily address. Quotations are reported in the participants' own words; light editorial cleaning is indicated by ellipses. Themes supported by multiple participants are flagged as such, and themes supported by a single participant are reported as single-source findings with appropriate analytical caution.

## 5.2 Participant Overview

Eight recordings yielded eight distinct participants. The recording for P03 and P04 contained two participants in conversation; the remaining six contained one each. Table 5.1 summarises the participants and the substantive content of each recording. The seniority and exposure profile is varied by design: one director-level participant with AI-first delivery responsibility (P02), two participants with more than twenty-five years of experience (P03, P04), two mid-career specialists with deep AI-adjacent engagement (P06, P07), an insurance-sector consultant (P01), a comparatively earlier-career AI engineer (P05), and a senior industry representative from an established DevOps platform vendor (P08). Senior participants tended to engage at the architectural level and were the source of the strongest critical material; mid-career participants tended to engage at the workflow and adoption level; the vendor participant contributed a market-level and strategic perspective distinct from the practitioner accounts. Where the depth or direction of a theme varied with role or seniority, this is noted in the relevant theme. P08 is a vendor representative rather than a practising engineer, and the analysis treats their contributions as an industry perspective rather than as lived engineering experience; their material is drawn on for the pattern-emergence, boundary, adoption, and future-trajectory themes and is explicitly attributed as a vendor view wherever it is used.

## 5.3 Findings on Architectural Principles (SQ1)

Five themes bear on SQ1. Two corroborate the thesis's framing of the problem and the timeliness of its proposed response. One substantively challenges a core design choice. One surfaces an unresolved architectural question. One extends the system-specific-intelligence principle into a cross-functional frame the thesis had not foregrounded.

### 5.3.1 Theme 1: Epistemic Stress Is Recognised in Practitioners' Own Terms

The construct of epistemic stress is recognised by participants in their own language, framed as a routine rather than exceptional feature of contemporary software engineering at scale.

The clearest in-data formulation came from P01, who described it in the context of a large legacy codebase: *"Sometimes some issues come up that even people who have been there for nine years don't even understand how that particular part of the code ever existed… code is written faster than it is comprehended."* The framing

Table 5.1: Participants and recordings

| ID | Role | Years | Notes on recording |
|---|---|---|---|
| P01 | DevOps consultant, insurance | ~10 | Full substantive content |
| P02 | Director of Data and AI | 20 | Full substantive content |
| P03 | Front-end Tech Lead, fintech | 26 | Full substantive content; building a company-wide context layer |
| P04 | Back-end developer, fintech | ~25+ | In combined recording with P03; contributed on trust, retrieval, and stress-testing |
| P05 | AI Engineer | ~4.5 | Full substantive content |
| P06 | AI Developer, MarTech | ~10 | Full substantive content |
| P07 | AI/Data Engineer, manufacturing startup | ~6 | Full substantive content; parallel work on an enterprise semantic layer |
| P08 | Senior Technical Account Executive, DevOps platform vendor, Nordics | ~30 | Full substantive content; industry-vendor perspective |

is consequential because it grounds the construct in tenure and lifecycle observation rather than in researcher rhetoric; P01 is naming a routinely encountered phenomenon, not endorsing a researcher's coinage.

P03 traced the same phenomenon from the opposite direction. Their organisation moved from prompt engineering to systematic context engineering, and on to building a company-wide context layer, once context emerged as the binding constraint on what AI tools could do. Reflecting on the demonstration, P03 gave the construct its most compressed formulation: *"the problem of modern software isn't too much code, it's too little understanding… code is everywhere, but very few people actually understand what the code does."* P07 affirmed the construct directly when it was named during the session, describing applications *"we are not even understanding anymore"* and reporting parallel work on an enterprise-grade unified semantic layer in an unrelated domain.

### 5.3.2 Theme 2: The IIDP Pattern Is Emerging Independently in Industry

Three senior participants independently report building or observing infrastructures that match the IIDP pattern: global context layers fed to coding agents, semantic data platforms with knowledge graphs, and monorepo-spanning

Table 5.2: Theme support across participants. The vendor participant (P08) is listed separately, as their contributions are reported as a market-level view where they inform a theme.

| Theme | Contributing participants | P08 | SQ |
|---|---|---|---|
| T1 | P01, P03, P07 | – | SQ1 |
| T2 | P03, P06, P07 | vendor | SQ1 |
| T3 | P02, P04, P05, P07 | – | SQ1 |
| T4 | P01, P04, P05, P06, P07 | vendor | SQ1 |
| T5 | P06, P07 | – | SQ1 |
| T6 | P01, P02, P06 | – | SQ2 |
| T7 | P01, P03, P04, P05, P06, P07 | – | SQ2 |
| T8 | P01, P03, P05, P06 | vendor | SQ2 |
| T9 | P01, P02, P03, P06, P07 | vendor | SQ2 |
| T10 | P01 | – | SQ2 |

embedding-based context systems. This suggests the thesis's contribution is the principled articulation of an emerging pattern, not the discovery of a novel one. P03 offered the strongest endorsement of timeliness early in the session, describing parallel work at their own organisation: a company-wide global context layer with curated context across all microservices, exposed through an MCP server so that coding agents in Cursor and Claude fetch relevant code, and crucially aware of the relationships between microservices; which P03 characterised as *"kind of like the superpower."* P03 judged the thesis *"super valid"* and *"ahead of the time."* P07 framed the parallel implementation from a different angle, having set up a unified data platform with vector storage, a knowledge-graph semantic layer, RDF and SPARQL, and a human-in-the-loop validation flow for a manufacturing startup; on seeing the demonstration, P07's assessment was that the artifact represented *"enterprise production grade level"* of what they had been sketching. P06 offered a third instance, drawn from organisational adjacency: a sister company running fifteen teams on a monorepo had built a plugin that clones the repository and adds semantic search so that all teams can access context across the system. A vendor-side observation reinforced the same reading from outside the practitioner accounts. P08, a senior representative of an established DevOps platform vendor, characterised the surrounding market as crowded and fast-moving; incumbents and AI-native entrants alike converging on context and understanding as the central problem; and judged the thesis's framing to be aligned with where the industry as a whole is heading. As a vendor perspective this carries a different evidentiary weight from the practitioner accounts, but it corroborates the independent-emergence reading: commercial vendors and internal engineering teams alike appear to be converging on the same pattern.

### 5.3.3 Theme 3: Vector-Based Retrieval-Augmented Generation Is Contested

A substantive technical critique emerges across senior participants of pure vector-based retrieval-augmented generation as the retrieval substrate for an IIDP. The strongest in-data critique came from P02, who works at director level on an AI-first agentic engineering stack: *"I'm completely over this vector-based, embedding-based RAG. So I've kind of hit the limit of that a while ago."* P02 referenced a published paper on theoretical limitations of embedding models and characterised the underlying problem as asymmetric: query and chunks live in spaces with different distributional properties that vector similarity does not adequately bridge. The constructive alternative P02 offered is consequential; rather than embedding the corpus and matching queries against it, P02 advocated a pattern in which the LLM is given an explicit index of the repository and permitted to plan its own retrieval through agentic traversal. P05 voiced a structurally similar concern, noting that simple cosine similarity *"would [not] be enough"* and proposing structural anchors to constrain retrieval. P07's parallel-implementation work uses a knowledge graph with RDF and SPARQL alongside vector embeddings, supporting the convergence on hybrid architectures. P04 engaged the retrieval substrate from a complementary direction. Having confirmed during the session that the artifact's confidence scoring rests on cosine similarity, P04 proposed an explicit relevance-feedback mechanism; allowing a user to mark a retrieved result as irrelevant so that the system refrains from surfacing that context in future. The proposal is significant because it treats retrieval quality as something to be corrected through use rather than fixed at indexing time, and it converges with the constructive thrust of P02's and P05's critiques: the retrieval substrate of an IIDP should be adaptive and contestable rather than a static similarity computation.

### 5.3.4 Theme 4: Boundary and Scope as an Unresolved Architectural Question

Participants raised, independently and in different forms, the question of where the boundary of an IIDP's context should fall; per microservice, per team, per organisation, across organisations; and presented it as the principal open question the thesis design does not fully close.

P01 formulated the question most explicitly: *"What is the right line on how much context you plug into the semantic tooling?… I work in a big company with a lot of systems, but we have a pretty clear boundary on the systems that we are working on, and then there are other systems in the company that we don't work on. Is that sort of a natural way to draw the line also for this platform?"* P05 raised

the cross-repository formulation during the demonstration, asking how unified context behaves across separate microservice repositories where the context of each is constantly changing. P06 raised a per-developer-versus-per-company formulation, asking whether each developer would run their own instance or whether one instance would serve a company with hundreds of repositories. P07 introduced a trust dimension not visible in the others, noting that single-organisation deployment inherits implicit trust whereas multi-organisation deployment raises trust questions the architecture must explicitly address.

P08 introduced the same question in its commercial form, as a multi-tenancy and positioning decision: whether an IIDP should live close to the individual developer at the point of coding, or as a central platform that must then be scaled, secured, and made multi-tenant across an organisation. This casts P06's per-developer-versus-per-company formulation as a product-architecture choice rather than a deployment detail, underlining that the boundary question is unresolved at the level of where the platform should sit, not only how widely its context should reach.

P04 added a resource-bounded dimension through the artifact's stress behaviour. Probing how the platform handles ingestion at scale, P04 surfaced that ingesting a large external repository on an eight-gigabyte machine while running a local model exhausts memory, whereas the same machine completes ingestion when an external model API is used in place of the local model. The observation reframes boundary-drawing as partly a deployment-resource decision: the scope an IIDP can carry is conditioned not only by where the conceptual context boundary is set but by the compute envelope of the deployment; a constraint the artifact's RAM-tiered configuration acknowledges but does not dissolve.

### 5.3.5 Theme 5: Cross-Functional Knowledge as IIDP Affordance

P06 and P07 extend the value claim of the IIDP beyond engineering into product, design, and cross-team review functions: once the substrate exists, the same questions may be asked by non-engineering roles, and the platform's user surface broadens accordingly. P06 framed the cross-functional extension in terms of the broader shift in tooling already visible in their organisation: *"All roles within a tech company are kind of becoming like more generic. So my PM is now using Cursor Claude Code and my designer is doing the same. So if this would work for the entire team or the entire org or whatever, I think this would be very valuable."* P07 framed it differently, in terms of cross-team transparency on pull-request review, observing that centralised context would let sister teams see what each is working on rather than each *"hunting by themselves"*. The implication for the architectural principle

is that user surfaces and permission models should be designed for cross-role access from the outset rather than as a follow-on capability.

## 5.4 Findings on Impact (SQ2)

Five themes bear on SQ2. The first establishes the empirical baseline against which any impact claim must be evaluated. The remaining four examine specific dimensions of impact: trust, adoption, future trajectory, and an ethical surface that the developer-profiling capability creates.

### 5.4.1 Theme 6: Fragmentation as Lived Experience

Participants describe day-to-day workflows characterised by tab-switching, tool sprawl, and the cognitive cost of holding context across heterogeneous systems; fragmentation as the baseline rather than as a problem-to-be-solved. P01 quantified it in the language of workflow itself: *"Typically when I need to troubleshoot an issue that spans across multiple components, maybe between five to ten different places, I need to have open different tools and systems or documentation sources."* P06 framed it from the originating moment of awareness during their first on-call shift, with fifteen-plus tabs open across Grafana, CloudWatch, the terminal, and ancillary systems. P02 named the same phenomenon at director level, identifying DataDog, AWS, and LogFire as scattered across *"five, six different places"* that together constitute the fragmented system the IIDP proposition addresses.

### 5.4.2 Theme 7: Trust Is Graded by Consequence and by Data Residency

Trust in the platform's outputs is not a single dial.

P01 articulated a consequence-graded view, distinguishing low-stakes outputs that admit manual verification from high-stakes outputs that demand it: *"If you get the report on a database error and the suggested fixes to do some form of migration or truncation of tables and so on, then of course you're going to definitely want to project it."* P05's framing made data residency the primary trust precondition, asking whether the vector database belongs to the deploying organisation or the platform vendor, and noting that absence of cryptographic protection on a vector store would itself breach trust. P06 surfaced a structural concern: the unified substrate creates a single point of failure for proprietary code and context, raising supply-chain-attack risk. P07 added a topology dimension; single-organisation deployment inherits trust that multi-organisation deployment must explicitly construct.

The fintech participants grounded the data-residency precondition in a regulated-industry context, where obligations under the GDPR and sector-specific regimes such as financial-inspection rules make the location and handling of code and operational data a compliance question rather than a preference. P04 identified keeping data in-house as a primary trust requirement, framed against the unresolved question of whether external LLM services can be trusted with proprietary code. P03 extended the point to its compliance form: operating under a banking licence, the ability to state that data remains local; and, beyond that, that the platform carries a recognised certification; functions as what P03 called a *"green card"* for adoption. The same regulatory framing surfaced a governance affordance: P03 described observability obligations under financial-inspection rules requiring that no branch or pull request be created without a linked tracking ticket, and identified automatic creation of such tickets as a concrete way the unified substrate could serve compliance rather than merely coexist with it.

### 5.4.3 Theme 8: Adoption Is Gated by Integration Breadth and Friction

The primary adoption barrier across participants is the operational cost of integrating the platform into a heterogeneous existing toolchain, rather than architectural disagreement with the IIDP proposition. P01 noted that GitHub support *"gets you pretty far"* but that documentation sources, observability platforms, and operations systems together constitute *"a wild range of combinations that you probably have to test out and try."* P05 specified the friction threshold directly: integration must be reducible to a single click to authenticate each connected system, because *"if you make the integration easy, it might get adapted quickly."* P06 surfaced an adjacent cultural factor; tool fatigue, the exhaustion engineers feel at the cadence with which new tools demand attention, regardless of merit.

P03 supplied the theme's most developed account, in two parts. The first is integration breadth as the decisive multiplier: P03 stated plainly that integrating with third-party tools *"that's the superpower,"* giving concrete instances; automatic creation of tracking tickets, connection to the team's error-monitoring and dashboards, and notification into the team's communication channels; and judged that such integration *"makes it ten times more valuable."* The second is cultural: P03 described adoption as fundamentally a culture shift, recounting more than a year of resistance; *"I saw the hate in the person's eyes"* on first demonstrating an AI coding tool; before a dedicated AI team was formed. P03 also named a friction condition the theme had not foregrounded: automation as a precondition for use at all. In P03's words, *"no developer is going to use this if you have manual steps."*

The vendor participant offered a more sceptical reading, which the analysis records as a distinct, market-level view. P08 judged that much of the capability the IIDP proposes is already present, in some form, across existing platforms and the wave of AI-native entrants, and that the space is crowded enough to make its twelve-month shape hard to predict. As a perspective from a competing vendor this is not a neutral assessment, and it is reported as such; but it sharpens the adoption finding. The barrier is not only integration breadth and tool fatigue but differentiation: a new IIDP enters a saturated field. P08's constructive advice was to win adoption by going deep in a narrow niche; specifically pull-request and code review; rather than broad.

### 5.4.4 Theme 9: Future Trajectory: Skill Convergence, Role Shift, Agent Substrate

Participants converge on a future trajectory in which DevOps as a discipline shrinks into the streams it enables, software-level concerns become increasingly automated, and the human role shifts upward toward goal-setting, business logic, and architecture. P02 framed it most strongly: *"In five years, the software is a solved problem. Like you don't even have to think about it. So it's more like our jobs is mostly like defining the goals, defining the architecture, defining the business logic."* P01 framed the same trajectory in terms of compression; DevOps becoming a smaller part of the lifecycle as specifications and context become the binding inputs to AI-assisted delivery. P07 framed it as skill convergence: every software and data engineer becoming, in effect, an AI engineer building or consuming semantic layers for AI agents. P06 introduced a counter-current, warning against silver-bullet posture and autonomous mega-agents, and surfacing a wellbeing dimension that no other participant raised: *"Everyone is doing a lot and that will lead to burnout… that comes at a price, especially like mental price."* The role-convergence strand drew direct support from the recovered sessions. P03 anticipated that tooling of this kind will diffuse role boundaries; developers taking on operations work and the reverse; with engineers becoming *"more builders"* but with *"more niche, topic-specific specialities,"* and located the IIDP's organisational home in a developer-experience function partnered with DevOps. P07 stated the convergence even more strongly: *"every software engineer is nothing but an AI engineer,"* with AI engineering becoming a baseline skill that practitioners apply within their own niche rather than a separate discipline. P08 extended the trajectory in a direction the other participants did not, proposing that the IIDP be designed not around a human user but around an AI-only flow; a delivery pipeline in which the platform exposes interfaces to AI agents throughout, with humans retained in the

loop only at selected points. In this framing the long-run design target is the progressive removal of the human-facing user interface in favour of an agent-facing substrate. This is a vendor's strategic projection rather than a validated outcome, but it connects directly to the agent-substrate strand of this theme: if the IIDP's real users become agents rather than engineers, the unified context layer is better understood as infrastructure for those agents than as a destination for human queries.

### 5.4.5 Theme 10: Developer Profiling Carries an Ethical Surface (single-source)

The platform's developer-residency profiling capability is recognised as useful but raises a surveillance affordance: the same fingerprints that surface institutional knowledge also surface activity patterns visible to managers. The theme is supported by a single participant and is reported as a single-source finding. P01 raised the concern unprompted: *"There is sort of a mild ethical dilemma there that you can actually also, if you are a role manager, you can actually also see who is working on what, how many commits are they producing and so on."* The framing was specific; the issue is not that the information is wrong, but that its raw form invites management misuse. The constructive suggestion that followed was to present commit counts as spans or visual encodings rather than as raw numbers.

# 6

# Discussion

This chapter steps back from the empirical findings and asks what they imply for the thesis's design claims. It is organised around the themes from Chapter 5 that most directly challenge, qualify, or extend the propositions developed in Chapters 2 and 4. The ten themes divide by the kind of engagement they demand. Three carry implications strong enough to reshape a design claim and are treated in full: the retrieval substrate (Theme 3, Section 6.1), boundary and scope (Theme 4, Section 6.2), and the developer-profiling surface (Theme 10, Section 6.3). Trust (Theme 7) and adoption (Theme 8) are treated as conditions of use in the answers to the research questions rather than as separate discussion sections. Section 6.4 reframes the predictive ambition in the AIOps literature in light of the future-trajectory theme (Theme 9). The remaining themes are confirmatory of the thesis's framing rather than corrective of its design: epistemic stress recognised in practitioners' own terms (Theme 1), fragmentation as lived baseline (Theme 6), the independent emergence of the pattern (Theme 2), and the cross-functional affordance (Theme 5) together establish that the motivating condition and the artifact's reach are recognised, and are carried directly into the answers to the research questions in Chapter 7 rather than re-interpreted here. Section 6.7 reports the study's limitations.

## 6.1 The Retrieval Substrate Question

The strongest in-data challenge to the artifact's architecture came from senior participants on the retrieval substrate (Theme 3): a convergent practitioner judgement that pure vector retrieval is reaching the edge of what it can responsibly

carry, with the constructive alternative being agentic retrieval guided by an explicit codebase index. This engages Decision 1 of the artifact (Section 4.4) at its most exposed flank. The artifact's actual pipeline is not pure vector RAG; it composes vector retrieval, LogConcepts, and knowledge-graph traversal with cross-encoder reranking. The critique therefore partially confirms a direction rather than refuting one. The critique is not satisfied by the hybrid pipeline alone, however: P02's alternative is more radical, abandoning the bi-encoder substrate (the approach in which queries and stored content are embedded separately and matched by vector similarity) at the foundation in favour of agentic planning over a structured index.

The two approaches differ functionally as well as architecturally: bi-encoder retrieval matches a query against precomputed representations in a single pass, while agentic retrieval lets the model plan a sequence of tool-mediated lookups over an explicit index at query time, trading latency and cost for adaptivity. A reflexive note also attaches to the source of the critique: P02's professional stack is built on the agentic pattern, so the judgement reflects committed practice rather than a neutral comparison, and it is weighed here as a strong practitioner position rather than as adjudicated evidence.

This is precisely the boundary of the RAG paradigm reviewed in Section 2.4: Lewis et al. and the Sentence-BERT lineage ground generation in dense-vector similarity, while the reference-architecture and integration-failure work already signals that grounding mechanism as the contested layer rather than a settled one [17, 31, 30, 33]. The substantive contribution of Theme 3 to the IIDP research programme is therefore the separation of the principle from its current realisation. The IIDP design principles do not commit to any particular substrate: Principle 1 requires a single retrieval semantics across modalities, not that the retrieval mechanism be bi-encoder vector similarity. A future iteration in which the bi-encoder is replaced by agentic planning over a structured index would still instantiate the principle. Future work should treat retrieval-substrate choice as an empirically contestable design parameter rather than a fixed commitment. This convergence of practitioner judgement with the recurring failure points documented for production retrieval-augmented generation systems [33] indicates that the retrieval-substrate question is open in the literature as well as in practice, and should be treated as such rather than as a settled implementation detail.

## 6.2 Boundary and Scope as an Open Architectural Problem

The boundary-and-scope question (Theme 4) surfaced in four forms across participants, from per-repository scope to inter-organisational trust topology. The

artifact provides a technical handle through the multi-repository scoping mechanism described in Section 4.2, but the deeper question, how the boundary should be drawn in the first place, is not one the artifact can answer. P07's framing makes the trust-topology dimension inseparable from the architectural one: a unified-context platform that crosses organisational trust boundaries inherits the trust problem of inter-organisational data sharing. The thesis does not resolve the boundary question; it contributes by stating it as a first-class architectural concern that the IIDP design programme must engage in subsequent work. The boundary question echoes, at the level of context rather than ownership, the team-topology boundaries that Skelton and Pais [4] treat as fundamental to platform design; the IIDP raises the same question for the semantic representation, and the literature on platform engineering offers no ready answer to it.

## 6.3 Governance, Ethics, and the Developer-Profiling Surface

The developer-profiling ethical surface (Theme 10) requires a design response. The concern is not that the information is wrong but that its raw form invites managerial misuse. Three design responses follow: aggregation by default, so that profiles surface patterns of contribution at a granularity that resists single-person fingerprinting wherever practical; access controls aligned with intent, distinguishing profiles intended for onboarding and review-assignment from those visible to performance-management workflows; and transparency to the profiled, so that engineers whose contribution patterns are represented can inspect and contest the representation. The current artifact does not implement these mitigations; they are flagged as design commitments for future iteration. A related governance surface, raised by the unified substrate itself rather than by profiling, is access control: a platform that concentrates code, logs, and history into one queryable memory also concentrates access to them, and reconciling that concentration with the principle of least privilege (role-scoped retrieval, so that a query answers only from context its asker is entitled to see) is an open design problem the current artifact does not address. This returns the developer-profiling surface to the definition of governance adopted in Section 2.1: governance as the continuous, auditable verification of how the delivery process operates. A capability that makes contribution patterns continuously visible is, on that definition, a governance mechanism, and it must therefore be designed under the same constraints of accountability and transparency rather than treated as an incidental analytics feature. This is consistent with the critique in Section 2.1 that current IDPs implement governance as checkpoint-based gates [8, 23]; a continuously visible contribution surface is

governance in the embedded sense the thesis argues for, and inherits the same accountability obligations.

## 6.4 From Predictive Ambition to Proactive, Continuous Intelligence

The literature identifies the prevailing reactive posture of software operations as a gap in predictive, system-specific intelligence, particularly in the AIOps call for a shift from isolated detection toward holistic prediction and prevention [3, 10, 28]. The evaluation data engage this ambition, but they reframe it. No participant endorsed prediction in the forecasting sense; instead, several independently described the property they valued as *proactive, continuously updated* intelligence rather than the anticipation of future failure. P08 articulated this most directly, describing the goal as backend processes that run continuously and guide the user proactively, in contrast to the manually triggered workflows that characterise current AI tooling, where assistance arrives only at the end of a task and is then limited. P03 described the same property from an implementation angle: agents attached to every merge to the main branch, generating change fragments that are continuously folded back into the context layer with no manual step because, in P03's words, no developer will use a system that has manual steps. P07 drew the distinction within the artifact itself, separating the reactive auto-heal capability, which acts only on error logs that already exist, from the intelligence engine's intended role of surfacing what should be attended to before an incident escalates.

This reframing matters for how the predictive ambition should be carried forward. The artifact does not implement prediction: PR risk scoring is retrospective pattern-matching against historical bugs, and auto-heal is reactive by construction. What the four principles do support is the substrate on which proactive behaviour can be built. Continuous, ambient governance (Principle 4) specifies that code, telemetry, and queries are continuously ingested, and a substrate that is always current is the precondition for any agent that acts ahead of an incident rather than after it. The predictive, preventive posture the research problem calls for is therefore best understood not as a claim the present evaluation substantiates, but as a direction the principles make reachable: the move is from *prediction* as a forecasting goal toward *proactive, continuous intelligence* as a design property, and the IIDP design programme should pursue it as future work on top of the principles rather than as a capability of the current artifact.

## 6.5 Critical Reflection on the Artifact

The discussion so far has engaged the critiques participants raised. Several further limitations of the artifact warrant acknowledgement independently of the evaluation data, because they bear on how far the instantiation can be taken as evidence for the principles.

The first concerns embedding quality and the loss of code semantics. Representing a function as a single dense vector necessarily discards structure: control flow, type relationships, call hierarchies, and the distinction between what code does and what its comments claim it does are all compressed into one similarity geometry. The Jina code-embedding model mitigates this with code-aware training and a long context window, but the representation remains lossy, and retrieval quality degrades where meaning depends on structure the embedding does not encode; a known failure class for production retrieval systems [33]. The participant critique of bi-encoder retrieval (Theme 3) is, in this light, a critique of exactly this loss.

The second concerns tacit knowledge. The platform externalises what is codified; commit history, code, logs; but the epistemic-stress framing itself (Section 2.6) implies that a substantial part of architectural understanding is tacit and never enters any artefact the platform can ingest. RootOps can therefore narrow the gap between artefacts and understanding, but it cannot capture knowledge that was never written down; the developer-profiling capability approximates who holds such knowledge, not the knowledge itself.

The third concerns scale and maintenance. The evaluation target is a single repository of modest size; the behaviour of a shared embedding space, and of cross-encoder reranking latency, at the scale of hundreds of repositories is undemonstrated, and P04's stress observation (Theme 4) already shows resource limits at the edges of a single-host deployment. The knowledge graph adds a maintenance burden of its own: causal edges promoted from observed correlations can become stale or wrong as the system evolves, and the artifact currently has no mechanism for demoting or expiring edges, so graph quality is expected to decay without curation. These are not incidental engineering gaps; they are open problems any production instantiation of the pattern would need to resolve, and they qualify the strength of the feasibility claim accordingly.

## 6.6 Ethical and Societal Implications

The conclusions of this study, that AI may serve as foundational platform infrastructure and that doing so concentrates lifecycle context into a single semantic

memory, carry ethical and societal implications that extend beyond the artifact itself and that the IIDP design programme should engage explicitly.

Three ethical implications follow from the conclusions. The first is the developer-profiling surface discussed in Section 6.3: a capability that makes contribution patterns continuously legible invites managerial misuse, and its mitigation (aggregation by default, intent-aligned access control, and transparency to the profiled) is an ethical obligation rather than an optional refinement. The second is concentration risk. A platform that unifies code, logs, and history into one queryable store also concentrates the consequences of a breach or a mis-scoped query; participants perceived this single point of failure as a supply-chain and least-privilege concern (Theme 7), so access governance is part of the design's ethical footing rather than incidental to it. The third is accountability for consequential automation: when the platform proposes remediations, responsibility for an accepted fix must remain locatable with a human, which is why the artifact gates auto-heal behind explicit approval.

Two societal implications also follow, both perception-based rather than measured. First, several participants expected the human role to shift upward toward goals, architecture, and business logic, and one (P06) named a wellbeing cost (the "mental price") of the pace of change; a platform that accelerates delivery may amplify this pressure rather than relieve it. Second, a reflexive caution attaches to the study's own premise: a platform built to relieve epistemic stress could, if its outputs are over-trusted, deepen reliance on AI and erode the institutional understanding it is meant to restore. Set against these, the local-first deployment that participants in regulated settings treated as a precondition lowers one societal risk, keeping proprietary code and operational data under the deploying organisation's control and supporting obligations under regimes such as the GDPR. These implications are not resolved by the present demonstration-based evidence; they are stated as first-class concerns for the design programme and for any organisation adopting the pattern.

## 6.7 Limitations

Four limitations should be acknowledged. First, the study draws on eight interview recordings yielding eight distinct participants; the sample was purposively selected for relevant expertise but is not statistically generalisable, and senior voices are over-represented relative to the broader engineering workforce. One participant (P08) is an industry-vendor representative rather than a practising engineer; their contributions are treated throughout as a market-level perspective and are attributed as such wherever they inform a theme. Second, the author is both the

designer of the artifact and the researcher conducting its evaluation; this dual role cannot be eliminated and is addressed through the mitigations described in Section 3.5. Third, the thematic analysis was conducted manually by the researcher following the procedure set out in Section 3.3; reflexive thematic analysis is interpretive by design, and a different analyst could have clustered the same codes differently, although the verification of every theme and quotation against the transcripts constrains this. Fourth, the evaluation is demonstration-based rather than longitudinal: it captures considered judgements formed during a demonstration session and structured interview, and does not capture sustained-impact effects of using the artifact over months. Demonstration-based DSR evaluation is appropriate for feasibility and principle-validation rather than for sustained-impact claims [42].

Table 6.1 summarises these limitations as validity threats in the conventional vocabulary, alongside the corresponding trustworthiness criterion (Section 3.5) and the mitigation applied.

Table 6.1: Validity threats, corresponding trustworthiness criteria, and mitigations

| **Threat** | **Criterion** | **Mitigation** |
|---|---|---|
| Researcher bias (designer, demonstrator, and sole analyst are the same person) | Confirmability | Open-source release of artifact and target for independent inspection; verbatim quotation verification against transcripts; critical and contradicting material coded as first-class; reflexivity statement (Section 3.3) |
| Demonstration bias (researcher-operated sessions may steer attention toward the artifact's strengths) | Credibility | Fixed structured protocol across all sessions; participants invited to interact and probe freely; evaluated baseline held constant across sessions; critical questions (retrieval, confidence scoring, scale) recorded and reported |
| Purposive sampling (senior, AI-adjacent participants over-represented) | Transferability | Selection criteria stated explicitly; spread across seven organisational contexts; vendor perspective separated and attributed; thick description of roles in Section 5.2 |
| Limited external validity (single demonstration target; no longitudinal deployment; perception-based evidence) | Transferability / Dependability | Claims bounded to feasibility and practitioner-judged value; impact reported as perceived rather than measured; longitudinal quantitative evaluation stated as future work (Chapter 7) |

# 7

# Conclusions

## 7.1 Answering the Research Questions

The thesis answers the main research question in two ways. First, it presents RootOps, the artifact described in Chapter 4: a working, openly released, locally deployable monorepo that instantiates four IIDP principles. Second, it interprets the expert evaluation reported in Chapter 5 and discussed in Chapter 6: a set of practitioner judgements that validate, challenge, and extend the design.

**SQ1** is answered by four principles: a shared semantic substrate across modalities, augmented by a knowledge graph; system-specific learning over a bounded scope; AI as the foundational layer rather than an addition; and governance as a continuous, embedded, provenanced, and locally resident property. The evaluation supports these principles in three ways. Practitioners recognised the fragmentation problem in their own work; the IIDP pattern appears to be emerging independently across contexts; and the retrieval-substrate critique constrains the implementation without refuting the principles. At the same time, the boundary-and-scope question remains unresolved.

**SQ2** is answered across three impact dimensions, with the caveat that the evidence is based on practitioner perception rather than measured outcomes. For software delivery, participants saw the artifact as most useful for incident response and cross-modal investigation, especially where it could reduce time-to-diagnosis. For developer experience, the main perceived benefit was cognitive: a single place to ask cross-cutting questions. For governance, participants treated continuous provenance, embedded risk visibility, and local-first residency as meaningful responses to checkpoint-based governance, while developer-profiling ethics

and single-point-of-failure risks remain open concerns. Whether the platform improves these dimensions in deployment is a question for the longitudinal evaluation identified as future work.

## 7.2 Contributions

The contribution of this thesis is not the invention of the Intelligent Internal Developer Platform pattern. The evaluation suggests the opposite: three participants reported building or observing infrastructures that match the pattern (Theme 2), and the vendor participant judged that related capabilities already exist across the tooling market. What the literature lacks is not the practice itself, but a clear articulation of it.

AI is already applied to software delivery through editor-integrated assistants such as GitHub Copilot and agentic CLIs [29, 11], operational AIOps systems for detection and correlation [10, 3], and retrieval-augmented systems that ground generative models in external knowledge [17]. These approaches usually treat AI as a capability added at a specific point in the lifecycle. The pattern articulated here is different: AI becomes the foundational layer through which the platform is queried and operated, over a single semantic representation spanning code and operational data. The thesis contributes a principled, teachable, and comparable formulation of that pattern, together with an empirical account of how senior practitioners evaluate it.

The contributions fall into three categories conventional in Design Science Research [41, 42]. The **artifact contribution** is RootOps, the open-source AI-native IDP at `github.com/Intelligent-IDP/rootops`. Concretely, the artifact is a working software system: a FastAPI ingestion and retrieval service, a Next.js dashboard, and a PostgreSQL/pgvector semantic memory holding embeddings, LogConcepts, and a knowledge graph, deployed as a single Docker Compose stack that anyone may clone, run, and inspect. It instantiates the four IIDP principles end-to-end against a real codebase, and it is, to the author's knowledge, the first openly released instantiation of the pattern as a whole. The **empirical contribution** is the structured account of how eight experienced practitioners evaluated the artifact: ten cross-cutting themes, including the practitioner recognition of epistemic stress in their own terms, the independent emergence of the IIDP pattern across unrelated contexts, and the identification of integration breadth and cultural tool fatigue as the principal adoption barriers. The **theoretical contribution** is the articulation of the IIDP pattern itself. The principles state what must remain stable, while leaving room for future versions to use a different retrieval substrate or deployment topology.

The distinction from prior work can be stated directly. Where the literature often treats operational difficulty as a metrics problem (delivery performance) or a tooling problem (fragmented tools), this thesis frames it as a problem of *understanding*: the gap between code produced and code comprehended. It names that gap as *epistemic stress* and contributes a teachable formulation of the architectural pattern that addresses it, evaluated with the practitioners who would adopt it. That formulation, not the discovery of the pattern, is the contribution.

## 7.3 Implications and Future Work

Three implications follow for practice, each grounded in the evaluation rather than in measured outcomes. First, participants in regulated and proprietary-code settings treated local-first deployment as a precondition rather than a feature. Second, participants graded their trust by the consequence of the output: they expected explicit verification steps for high-stakes recommendations such as proposed fixes, and accepted lighter checking for low-stakes ones such as search results. Third, participants judged integration breadth, not version-control support alone, decisive for whether the platform is experienced as complete.

Two implications follow for research. Retrieval-substrate choice should be treated as an empirically contestable parameter rather than a fixed commitment, and the drawing of context boundaries should be treated as a first-class architectural concern, studied together with the trust relationships it crosses.

Future work for the programme falls into four directions: controlled experimentation on retrieval substrates, using the artifact's pluggable retrieval architecture; longitudinal field evaluation, to produce the quantitative evidence on impact that this demonstration-based study could not; design research on boundary and trust questions, including federated IIDP topologies that span trust domains without unifying their substrates; and governance work on developer profiling, single-point-of-failure concentration, and consequence-graded trust.

## 7.4 Closing Reflection

The proposition this thesis advances is that AI should be foundational infrastructure for software delivery, not a feature added to it. The empirical evidence suggests that experienced practitioners, working independently across unrelated organisations and problem domains, are reaching for a similar architectural shape under the pressure of fragmented intelligence and epistemic stress. The thesis articulates that shape, instantiates it in a working artifact, and subjects it to structured expert evaluation that surfaces both endorsement and critique. The field is still unsettled,

and the IIDP design programme this thesis contributes to does not determine where it will settle; it provides infrastructure within which the question can be asked seriously.

# Bibliography


[1] Anil Kumar Veldurthi. “The Future of Platform Engineering: From DevOps to Internal Developer Platforms (IDPs)”. In: *World Journal of Advanced Engineering Technology and Sciences* 15.2 (2025), pp. 2408–2426.

[2] Nick Durkin, Eric Minick, and Chetan Gaikwad. *AI-Native Software Delivery: Proven Practices to Produce High-Quality Software Faster*. Sebastopol, CA: O’Reilly Media, 2025.

[3] Paolo Notaro, Jorge Cardoso, and Michael Gerndt. “A Systematic Mapping Study in AIOps”. In: *Service-Oriented Computing; ICSOC 2021 Workshops*. 2021.

[4] Matthew Skelton and Manuel Pais. *Team Topologies: Organizing Business and Technology Teams for Fast Flow*. IT Revolution, 2019.

[5] Puppet by Perforce. *2023 State of Platform Engineering Report*. Annual Report. Portland, OR, USA: Puppet, 2023.

[6] Humanitec and Gitpod. *State of Platform Engineering Report, Volume 3*. Whitepaper. Berlin, Germany: Humanitec, 2024.

[7] Gene Kim et al. *The DevOps Handbook*. IT Revolution Press, 2016.

[8] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution, 2018.

[9] Google Cloud. *2023 State of DevOps Report*. Annual Report. DORA / Google Cloud, 2023. URL: `https://cloud.google.com/devops/state-of-devops`.

[10] Yingnong Dang, Qingwei Lin, and Peng Huang. “AIOps: Real-World Challenges and Research Innovations”. In: *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 2019, pp. 4–5.

[11] Mark Chen et al. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint arXiv:2107.03374* (2021).

[12] Sumanth Amgothu, Anil Kumar Vedantham, and Sai Prakash M. Gowda. “Observability and AIOps in Cloud-Scale DevOps: Technologies, Architectures, Challenges, and Future Trends”. In: *Proceedings of the 38th Conference of FRUCT Association*. 2025.

[13] Nicole Harvey et al. *State of DevOps Report 2025: Lessons for Engineering Leaders*. Annual Report. DORA / Google Cloud, 2025.

[14] Ahmed E. Hassan et al. “Agentic Software Engineering: Foundational Pillars and a Research Roadmap”. In: *arXiv preprint arXiv:2409.06416* (Sept. 2024).

[15] Pankaj Prasad and Charley Rich. *Market Guide for AIOps Platforms*. Tech. rep. G00340492. Gartner Research, Nov. 2018.

[16] Adam Ndiaye and Xhensila Taraj. *Generative AI in Software Engineering: Transforming the Software Development Process*. Whitepaper. Accenture and DFKI, 2025.

[17] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.

[18] Yuhao Shao et al. “Are LLMs Correctly Integrated into Software Systems?” In: *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE ’25)*. 2025, pp. 1178–1190.

[19] Nicole Forsgren et al. *2025 State of AI-Assisted Software Development Report*. Annual Report. DORA / Google Cloud, 2025. URL: https://dora.dev/research/2025/.

[20] C. Peters and U. Usman. *AI-Native Software Development Lifecycles for End-to-End Automation*. ResearchGate. 2026.

[21] Cary Pillers and Manjunath Bhat. *Hype Cycle for Platform Engineering, 2025*. Gartner Report. Gartner Research, June 2025.

[22] Gartner. *Market Guide for Internal Developer Portals*. Tech. rep. Gartner Research, 2023.

[23] L. Cao et al. “Towards the Next Generation of Software: Insights from Grey Literature on AI-Native Applications”. In: *arXiv preprint* (2024).

[24] Len Bass, Ingo Weber, and Liming Zhu. *DevOps: A Software Architect’s Perspective*. Addison-Wesley Professional, 2015.

[25] N. Sato and Y. Ikawa. “The Impact of DevOps Practices on Software Quality and Performance: An Empirical Study”. In: *Journal of Software Engineering Research and Development* 7.1 (2019), pp. 1–15.

[26] A. Noda et al. “DevEx: What Actually Drives Productivity”. In: *ACM Queue* (2023).

[27] Nicole Forsgren et al. “The SPACE of Developer Productivity”. In: *ACM Queue* 19.1 (2021), pp. 20–48.

[28] Gartner. *Market Guide for AIOps Platforms*. Tech. rep. G00322184. Gartner Research, Aug. 2017.

[29] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. “Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models”. In: *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. 2022.

[30] Alessio Bucaioni et al. “A Functional Software Reference Architecture for LLM-Integrated Systems”. In: *Proceedings of the IEEE/ACM 22nd International Conference on Software Architecture Companion (ICSA-C ’25)*. 2025, pp. 1–5.

[31] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019.

[32] Alec Radford et al. “Robust Speech Recognition via Large-Scale Weak Supervision”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML ’23)*. Vol. 202. PMLR. 2023, pp. 28492–28518. arXiv: 2212.04356.

[33] Scott Barnett et al. “Seven Failure Points When Engineering a Retrieval Augmented Generation System”. In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering; Software Engineering for AI (CAIN ’24)*. Lisbon, Portugal, 2024, pp. 194–199. DOI: 10.1145/3644815.3644945.

[34] Michael Polanyi. *The Tacit Dimension*. Garden City, NY: Doubleday, 1966.

[35] Ikujiro Nonaka and Hirotaka Takeuchi. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. New York: Oxford University Press, 1995.

[36] James P. Walsh and Gerardo Rivera Ungson. “Organizational Memory”. In: *Academy of Management Review* 16.1 (1991), pp. 57–91.

[37] Karl E. Weick. *Sensemaking in Organizations*. Thousand Oaks, CA: Sage Publications, 1995.

[38] Edwin Hutchins. *Cognition in the Wild*. Cambridge, MA: MIT Press, 1995.

[39] John Sweller. “Cognitive Load During Problem Solving: Effects on Learning”. In: *Cognitive Science* 12.2 (1988), pp. 257–285.

[40] Eric Trist. “The Evolution of Socio-Technical Systems”. In: *Perspectives on Organization Design and Behavior*. Ed. by Andrew H. Van de Ven and William F. Joyce. New York: Wiley, 1981.

[41] Alan R. Hevner et al. “Design Science in Information Systems Research”. In: *MIS Quarterly* 28.1 (2004), pp. 75–105.

[42] Paul Johannesson and Erik Perjons. *An Introduction to Design Science*. Springer, 2014. DOI: 10.1007/978-3-319-10632-8.

[43] Kirsti Malterud, Volkert Dirk Siersma, and Ann Dorrit Guassora. “Sample Size in Qualitative Interview Studies: Guided by Information Power”. In: *Qualitative Health Research* 26.13 (2016), pp. 1753–1760. DOI: 10.1177/1049732315617444.

[44] Greg Guest, Arwen Bunce, and Laura Johnson. “How Many Interviews Are Enough? An Experiment with Data Saturation and Variability”. In: *Field Methods* 18.1 (2006), pp. 59–82. DOI: 10.1177/1525822X05279903.

[45] Monique Hennink and Bonnie N. Kaiser. “Sample Sizes for Saturation in Qualitative Research: A Systematic Review of Empirical Tests”. In: *Social Science & Medicine* 292 (2022), p. 114523. DOI: 10.1016/j.socscimed.2021.114523.

[46] Martyn Denscombe. *The Good Research Guide*. 6th ed. Open University Press, 2017.

[47] Virginia Braun and Victoria Clarke. “Using Thematic Analysis in Psychology”. In: *Qualitative Research in Psychology* 3.2 (2006), pp. 77–101.

[48] Yvonna S. Lincoln and Egon G. Guba. *Naturalistic Inquiry*. Sage Publications, 1985.

# Appendix A

## Consent Form

This appendix summarises the consent information provided to expert-interview participants before data collection. The consent process was designed to ensure that participation was voluntary, informed, and consistent with the ethical commitments described in Section 3.4.

### Study Information

- The study investigates the design and evaluation of an AI-native Internal Developer Platform called RootOps.
- Participation consists of an expert-interview session including a live artifact demonstration and semi-structured interview questions.
- The session may be recorded for transcription and analysis with the participant's permission.
- The study forms part of a master's degree project at the Department of Computer and Systems Sciences.

### Participant Rights

- Participation is voluntary and may be discontinued at any time.
- Participants may decline to answer any question.
- Participants may request clarification about the study before or during the session.

- Participants may contact the researcher after the session with questions or requests concerning their data.

## Data Handling and Anonymity

- Recordings and transcripts are used only for thesis analysis.
- Participants are anonymised using participant identifiers such as P01 and P02.
- Direct quotations may be used in the thesis only in anonymised form.
- Potentially identifying organisational or personal details are removed or generalised during analysis and write-up.

## Consent Statement

By participating, the participant confirms that they have received information about the study, understand the voluntary nature of participation, consent to the use of anonymised interview material for thesis analysis, and understand that any recording is used only for transcription and research purposes.

# Appendix B

# Interview Questions

This appendix reproduces the interview-question content of the evaluation protocol (version 2.0, May 2026). Each session followed five phases: introduction and consent, live demonstration of five scenarios, presentation of the four IIDP principles with invited critique, the semi-structured interview, and closing. Operational scripts and session checklists are omitted; the questions below constitute the data-collection instrument.

## Participant background

- What is your current role and how long have you been in it?
- Can you briefly describe your team's technology stack and delivery workflow?
- Does your organisation use an Internal Developer Platform? If so, which one and what does it cover?
- How many engineers are on your team or in your organisation?

## Demonstration scenario probes

Each of the five demonstration scenarios concluded with a reaction probe:

- *Semantic code search:* Does this kind of search problem — knowing something exists in the codebase but not being able to find it — come up in your work?

- *Cross-modal correlation:* When you investigate an incident today, how many tools do you typically have open? How long does it take to trace from an error to the relevant code?
- *PR risk scoring:* How does your team currently decide which PRs to review more carefully? Is that decision based on data or intuition?
- *Auto-heal:* What is your reaction to a system that proposes fixes autonomously? What would need to be true for you to trust it?
- *Developer profiling:* Has your team ever been caught off guard when someone who understood a critical system left or was unavailable?

## Reaction to the IIDP principles

- What is your reaction to this framing? Does the distinction between a platform that organises and one that understands resonate with your experience?
- Do these four principles capture what is missing from current platforms, or would you add, remove, or reframe any of them?
- Which principle feels most important to your team right now? Which feels furthest from reality?

## Concept probe: the motivating condition

- Does that term describe something you have experienced? Can you give an example?
- Where does the most critical understanding in your system currently live — in code, in documentation, or in people's heads?
- What happens when that understanding is unavailable — someone is on holiday, leaves the company, or is in a different time zone?

## Theme 1: Trust

- To what extent would you trust the platform's outputs — search results, risk scores, auto-heal suggestions — in your daily work?
- What would increase your trust? What would decrease it?

- Is there a difference in trust between the search results and the auto-generated fixes?
- How important is it that the platform explains how it reached a recommendation?
- Would you trust it more or less if it had been trained on your own system's history versus a generic model?

## Theme 2: Usefulness and developer experience

- Which of the capabilities you saw would be most useful to your team, and why?
- Are there capabilities you would not use? Why not?
- How does this compare to the tools you currently use for similar tasks?
- If you could only have one of the five capabilities, which would it be?

## Theme 3: Cognitive load and workflow

- How much of your time today is spent switching between tools and manually correlating information during incident investigation or code review?
- If a single platform could answer cross-cutting questions like "what caused this error?" in one query, how would that change your workflow?
- Which of the five scenarios addresses a problem that currently takes you the most mental effort?

## Theme 4: Governance

- How does your organisation currently handle governance in the delivery pipeline — compliance checks, audit trails, accountability for changes?
- Has your team ever reintroduced a bug that was fixed before, because nobody remembered the previous incident?
- When reviewing a PR, do you have visibility into how that change affects the rest of the system — downstream services, dependencies, historical patterns?

- If an AI-generated fix introduces a bug, who should be accountable — the developer who approved it, the platform, or the team?
- Do you see the PR risk scoring and auto-heal audit trails as governance mechanisms? Why or why not?
- Would continuous, automated governance be an improvement over checkpoint-based gates for your team?

## Theme 5: Adoption and organisational readiness

- What would need to be true for your organisation to adopt a platform like this?
- What are the biggest barriers you see — technical, organisational, or cultural?
- Would the self-hosted, privacy-first architecture matter to your organisation?
- Who would champion adoption — developers, engineering managers, or security and compliance?
- Do you see this as a replacement for existing tools or a layer on top of them?

## Closing: future trajectory

- Where do you think DevOps and platform engineering are heading in the next three to five years?
- Do you agree that the next evolution of platforms should focus on understanding rather than orchestration?
- What role do you see AI playing in software delivery beyond code generation and review?
- Is there anything about the current approach to AI in DevOps that concerns you?

# Appendix C

# Supplementary Resources

This appendix collects the openly available resources associated with this thesis. The source-code artifacts are public and may be inspected, cloned, or operated independently. The links were verified at the time of submission.

## C.1 Source Code Repositories

**RootOps** The artifact developed in this study: an open-source AI-native Internal Developer Platform, released under the Apache 2.0 licence.
`https://github.com/Intelligent-IDP/rootops`

**CORE Biz Manager** The demonstration target used in the expert evaluation: a real, publicly deployed Next.js business-management application against which RootOps was operated.
`https://github.com/theorjiugovictor/core-v1`

## C.2 Demonstration Target Deployment

**CORE Biz Manager** The deployed instance of the demonstration target application used during evaluation.
`https://usecoreapp.com/`

## C.3 Supporting Material

**Evaluation data** A shared folder containing the expert-interview recordings, transcriptions, and supporting analysis material that underpin Chapter 5.

Access is restricted to protect participant anonymity, in line with the ethical commitments described in Section 3.4, but may be provided by the author on request.
`https://drive.google.com/drive/folders/1-1K7EfNjICGBwu1fT2iQt-N9wZyMNpb8`

**Background article** A non-academic article by the author that introduces, in accessible terms, the problem of fragmented intelligence in modern software delivery that this thesis addresses.
`https://theorjiugovictor.substack.com/p/the-problem-with-modern-software`

# Appendix D

# Implementation and Tooling Details

## D.1 RootOps Repository Structure

Table D.1: Top-level structure of the RootOps repository

| Path | Role |
|---|---|
| `api/` | FastAPI service for ingestion, retrieval, synthesis, auto-heal, profiles, and graph maintenance |
| `web/` | Next.js dashboard |
| `docs/` | Documentation |
| `docker-compose.yml` | Production-style Compose configuration |
| `docker-compose.dev.yml` | Development Compose configuration |
| `Makefile` | High-level targets such as `make up`, `make dev`, and `make cloud-deploy` |
| `.env.example` | Environment-variable template |
| `LICENSE` | Apache 2.0 licence |

## D.2 Software Tools

Table D.2: Software tools used in the research

| **Activity** | **Tools** |
|---|---|
| Artifact development | Python and FastAPI; Next.js, React, TypeScript; PostgreSQL with pgvector; sentence-transformers (Jina v2 code embeddings, cross-encoder reranker); Docker Compose; pluggable language-model backends (Ollama, OpenAI, Anthropic, AWS Bedrock, Google Gemini) |
| Interview recording | Google Meet, recorded with local audio capture |
| Transcription | OpenAI Whisper, large-v3, MLX implementation [32], run locally on Apple Silicon |
| Anonymisation | Custom Python 3 script with word-boundary-safe regular-expression substitution; idempotent, with verification pass and pre-anonymisation backup |
| Thematic analysis | Markdown and `.docx` workbooks; manual coding and theme development following Braun and Clarke's six-phase approach |
| Thesis write-up | LaTeX on Prism; BibTeX/BibLaTeX for references |
| Reference management | Zotero |
| Diagramming | draw.io and Excalidraw |